

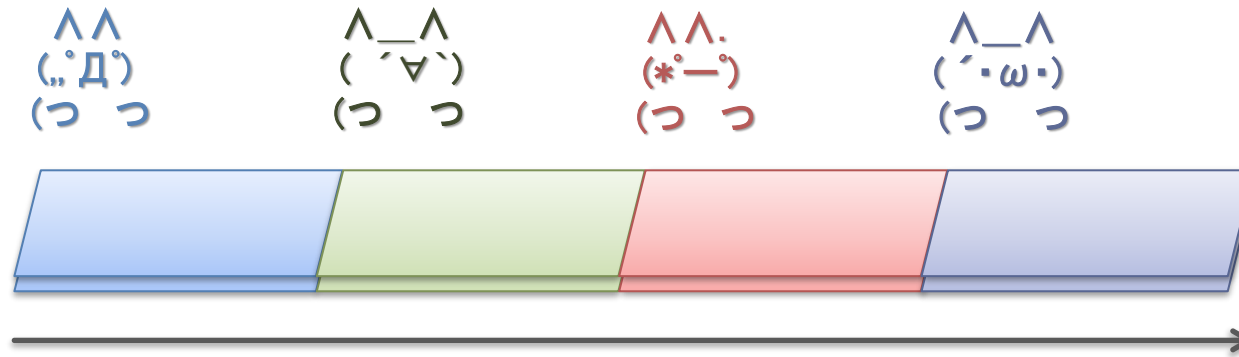
先進計算機構成論 02

東京大学大学院 情報理工学系研究科 創造情報学専攻

塩谷 亮太

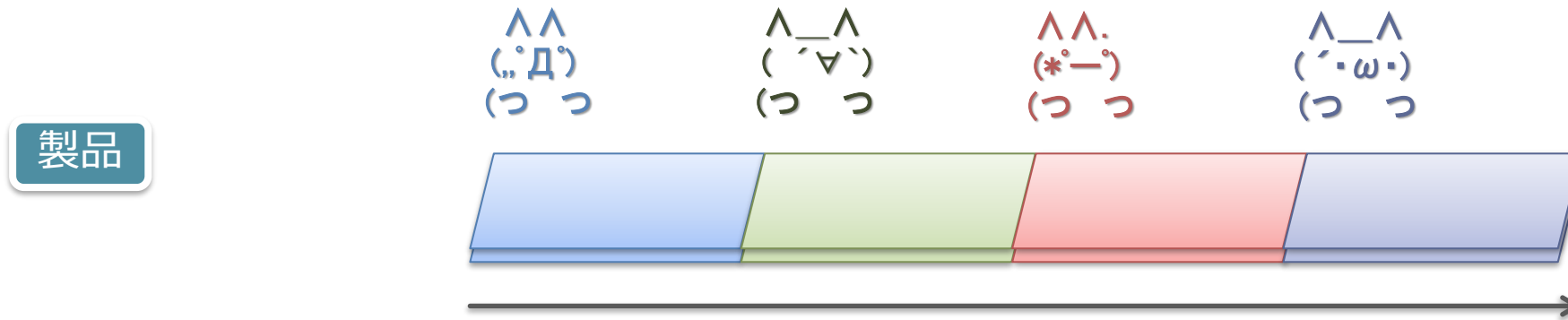
shioya@ci.i.u-tokyo.ac.jp

導入：工場のラインを考える



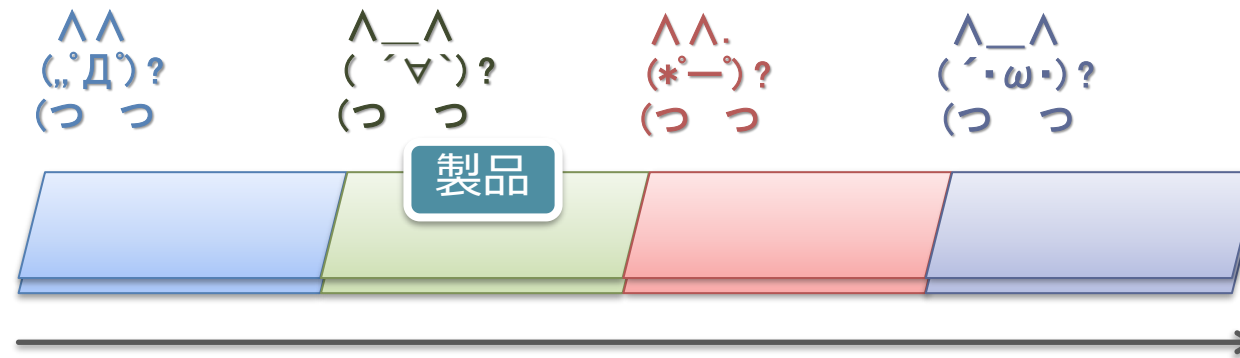
- ベルトコンベアのラインの上を製品が流れていく
 - 4 人の人が、それぞれの工程の作業をおこなって完成
- 上のように1つしか製品をながさないで、
 - 各人は他の人が作業している間はヒマ

導入：工場のラインを考える



- 実際の工場：複数の製品を同時に流す
 - 各工程を並列して処理することによりスループットを向上
 - さっきの4倍の速度で製品ができあがっていく
- これが 命令パイプライン
 - . . . の話を今日はしたいところではありますが

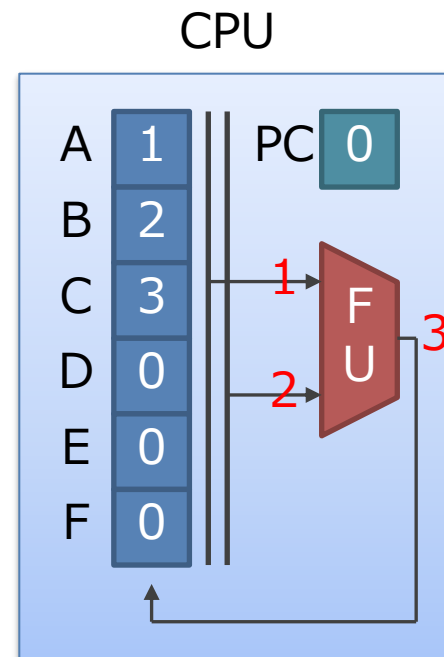
遅延に対する疑問



- 工場のラインの絵は，回路中を信号が伝搬されていくことのたとえ
 - 「製品」は，処理対象の信号（データ）
- そもそも「回路中を信号が伝搬される」のに「かかる時間」とは？
 - そりゃ，伝搬するのになんか時間はかかるのだろう
 - しかしそもそも回路が速いとか遅いとは，どういうことなのか？
 - 遅延は，「論理回路」の授業では理想化されていることが多い

回路に対する疑問

- 前回の講義：CPU の動作をふわっと説明
 - 「ここでもし命令が add だったら，加算する」
 - 「ここでもし命令が store だったら，メモリに書く」
- このような制御は，どのような回路で実現されるのか？



今日の内容：

- 命令パイプライン

- . . .ではなく回路と遅延, エネルギーの話をしてします

回路と遅延

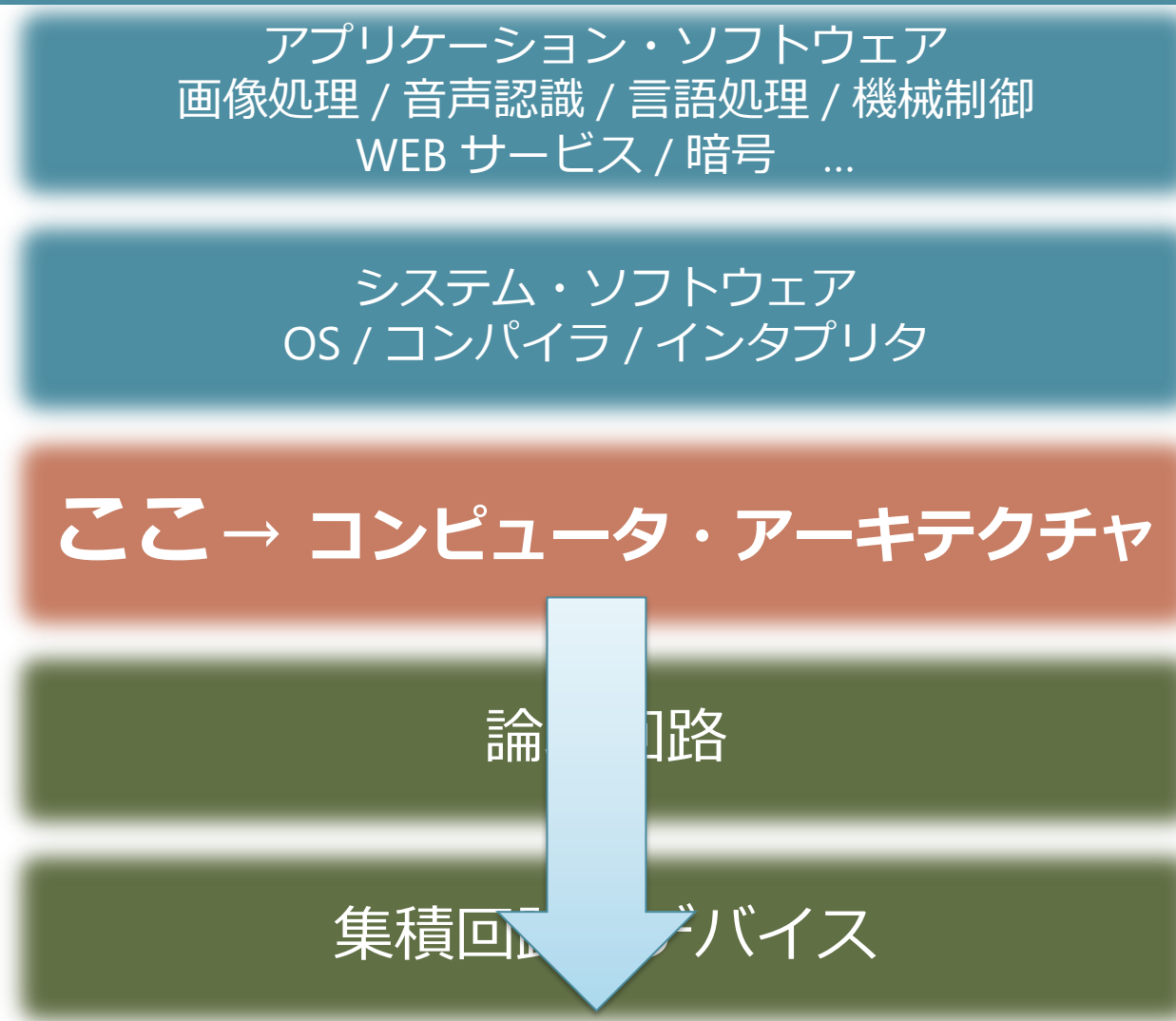
- 目的：これらの具体的なイメージを持つ
 - CPU の論理的な動作と、それを実現する回路の繋がり
 - それら論理回路の遅延や消費エネルギー
- 論理回路の復習から始めて説明
 - 論理回路
 - CMOS による実現
 - 遅延と消費電力がどのように決まるのか

情報の各分野の階層

- 前回は, 「C 言語で書かれたプログラムを動かすためには」という視点で上から迫っていきました



情報の各分野の階層



- 今回は、「コンピュータのハードを作るためには」という視点で、さらに下がっていきます

論理回路

組み合わせ回路と順序回路

1. 組み合わせ回路

- 出力が、現在の入力のみにより決定される論理回路

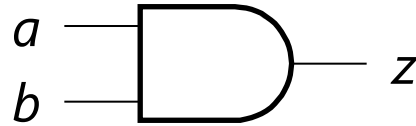
2. 順序回路

- 出力が、過去の入力（の履歴）にも依存する論理回路

組み合わせ回路の例：2入力論理ゲート

AND
(論理積)

MIL記号
MIL symbol



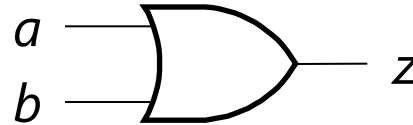
論理式
logic expression

$$z = a \cdot b$$

真理値表
truth table

a	b	z
0	0	0
0	1	0
1	0	0
1	1	1

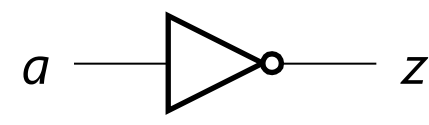
OR
(論理和)



$$z = a + b$$

a	b	z
0	0	0
0	1	1
1	0	1
1	1	1

NOT
(論理否定)



$$z = a'$$

$$z = \bar{a}$$

$$z = \neg a$$

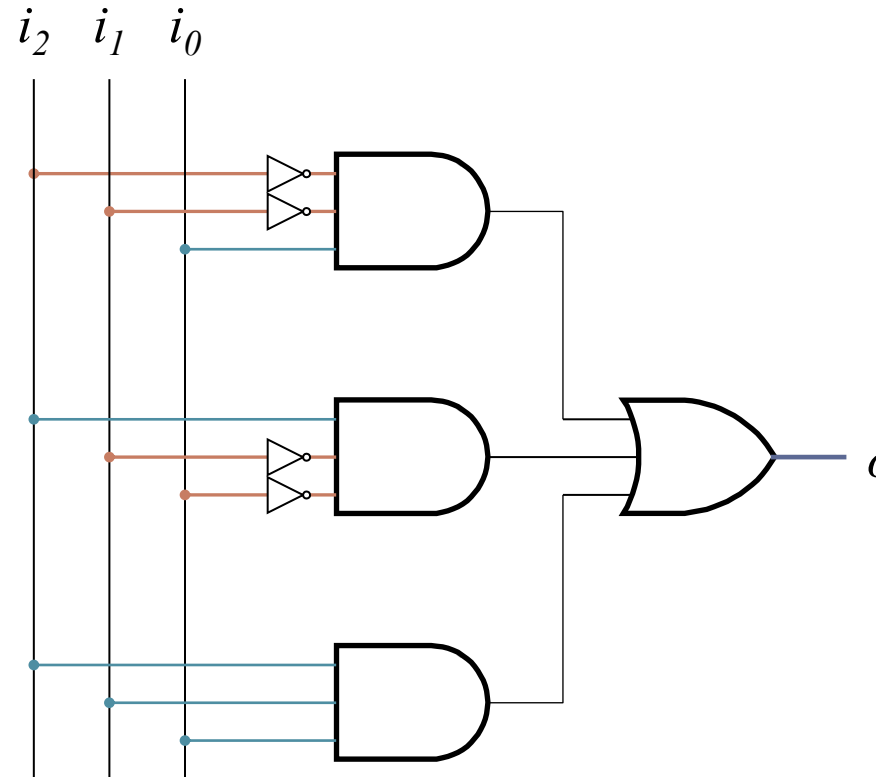
a	z
0	1
1	0

完全性 (Completeness, 完備性)

- 完全集合 (Complete Set) :
 - その組み合わせによって、すべての論理関数を表現できる論理関数の集合
- 完全集合の例
 - {AND, OR, NOT}
 - {AND, NOT}
 - {OR, NOT}
 - {NAND}
 - {NOR}
- たとえば、{AND, OR, NOT} を組み合わせると任意の論理関数が作れる
 - (証明は、数学的帰納法を使えばそんなに難しくない)

真理値表による表現と，積和標準系による回路

i_2	i_1	i_0	o
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1



$$o = i_2' i_1' i_0 + i_2 i_1' i_0' + i_2 i_1 i_0$$

■ 真理値表が与えられれば，

- {AND, OR, NOT} を使った積和標準系に機械的に置き換えできる
- つまり，{AND, OR, NOT} を使って対応する回路が生成できる

回路の例 : RISC-V の AND/OR/XOR 命令の演算

XOR: $x[rd] \leftarrow x[rs1] - x[rs2]$

0000000	rs2	rs1	100	rd	0110011
---------	-----	-----	-----	----	---------

OR: $x[rd] \leftarrow x[rs1] - x[rs2]$

0000000	rs2	rs1	110	rd	0110011
---------	-----	-----	-----	----	---------

AND: $x[rd] \leftarrow x[rs1] + x[rs2]$

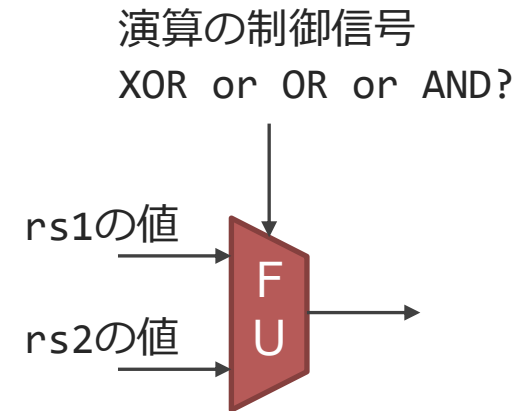
0000000	rs2	rs1	111	rd	0110011
---------	-----	-----	-----	----	---------

■ RISC-V の AND/OR/XOR 命令

- まず右端の opcode が 0110011 であれば, この3つのどれかということにする
 - 本当はほかの命令との識別がさらにあるが, ここでは忘れる
- 真ん中の3ビットの違いで識別する

制御の例 : RISC-V の AND/OR/XOR 命令

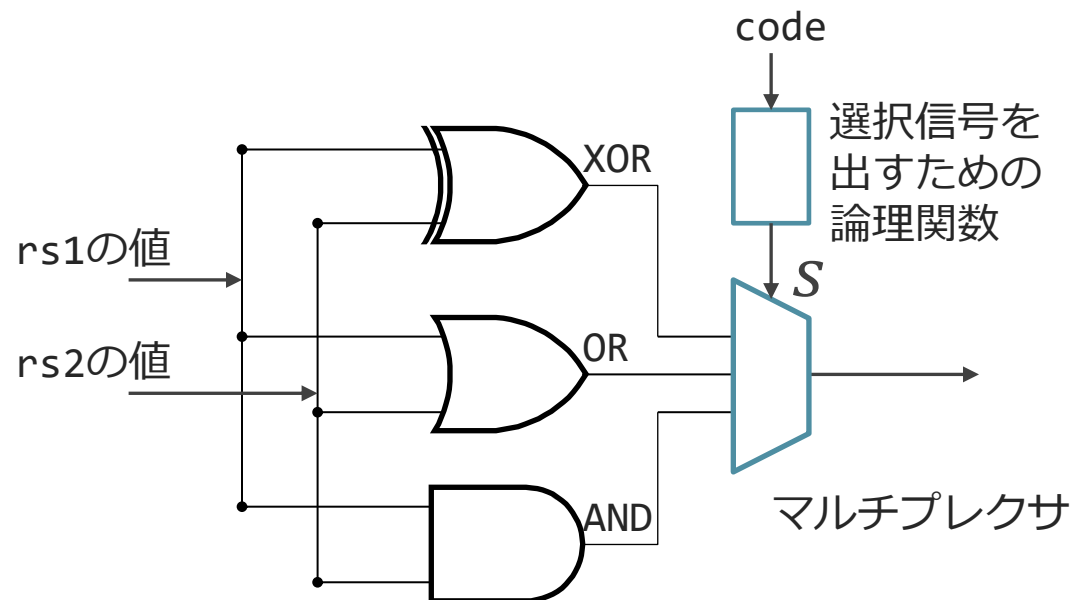
```
if code == 0b100:  
    rs1 xor rs2  
elif code == 0b110:  
    rs1 or rs2  
else:  
    rs1 and rs2
```



- 各命令の 3bit の code の違いに応じて
 - 演算器に, AND/OR/XOR 演算をさせることを考える

制御の例：RISC-V の AND/OR/XOR 命令

```
if code == 0b100:  
    rs1 xor rs2  
elif code == 0b110:  
    rs1 or rs2  
else:  
    rs1 and rs2
```

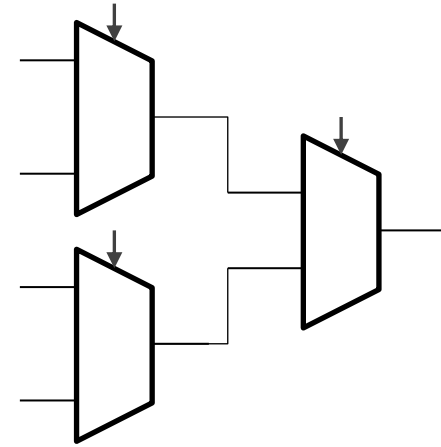
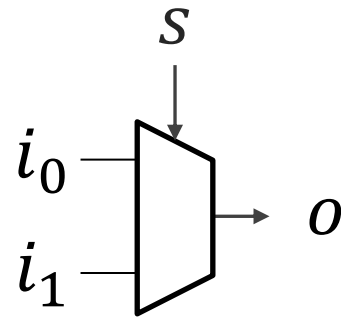


■ 典型的には,

- 各場合ごとの回路を用意して並列に配置
 - XOR, OR, AND ゲートを並べる
- 制御に従ってマルチプレクサで出力を選択

マルチプレクサ：複数入力から1つを選ぶ回路

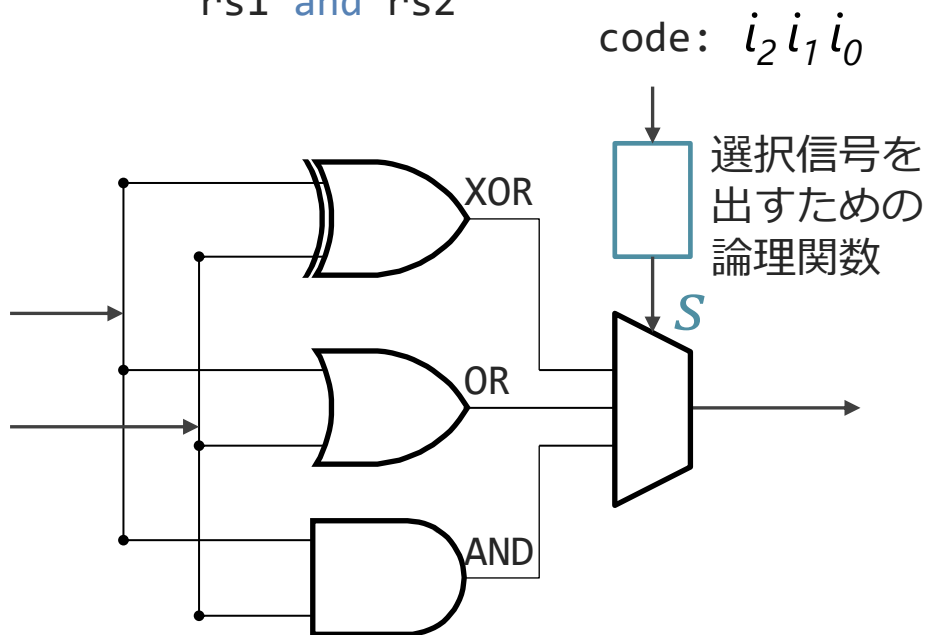
s	i_0	i_1	o
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1



- 以下により，回路が生成できる
 - 2 : 1 マルチプレクサは真理値表でかける = 回路が作れる
 - 多入力マルチプレクサは，カスケードすれば良い

選択信号を出すための論理関数

```
if code == 0b100:  
    rs1 xor rs2  
elif code == 0b110:  
    rs1 or rs2  
else:  
    rs1 and rs2
```

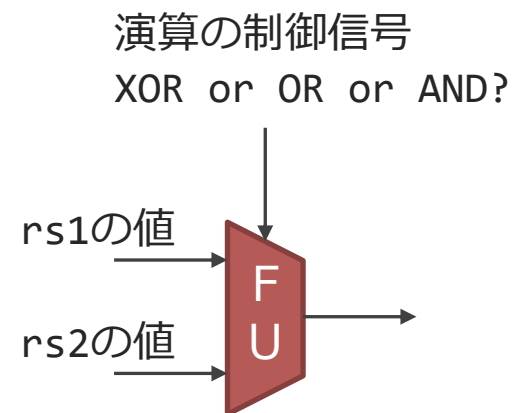


	i_2	i_1	i_0	S_1	S_0
	0	0	0	x	x
	0	0	1	x	x
	0	1	0	x	x
	0	1	1	x	x
XOR	1	0	0	0	0
	1	0	1	x	x
OR	1	1	0	0	1
AND	1	1	1	1	0

- 場合わけの制御は、そのまま真理値表にして回路を生成すればよい
 - XOR なら 00, OR なら 01, その他は 10

回路の生成のまとめ

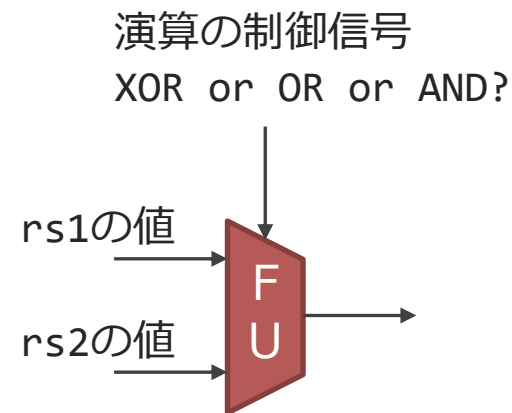
```
if code == 0b100:  
    rs1 xor rs2  
elif code == 0b110:  
    rs1 or rs2  
else:  
    rs1 and rs2
```



- 結局この手の, 「条件に応じて異なる演算を出力する回路」は,
 - 各場合ごとの回路を用意して並列に配置
 - 制御に従ってマルチプレクサで出力を選択
 - ……というように分解すれば, AND/OR/NOT 回路に落とし込める

回路の生成のまとめ

```
if code == 0b100:  
    rs1 xor rs2  
elif code == 0b110:  
    rs1 or rs2  
else:  
    rs1 and rs2
```



- code, rs1, rs2 を全て含む真理値表を作れば、そこから直接回路に落とし込むことも原理的にはできる
 - 表が大きくなりすぎて (2 の 3+32+32乗), 現実的には無理
 - もっと効率の良い表現方法で表現 (後述)

組み合わせ回路と順序回路

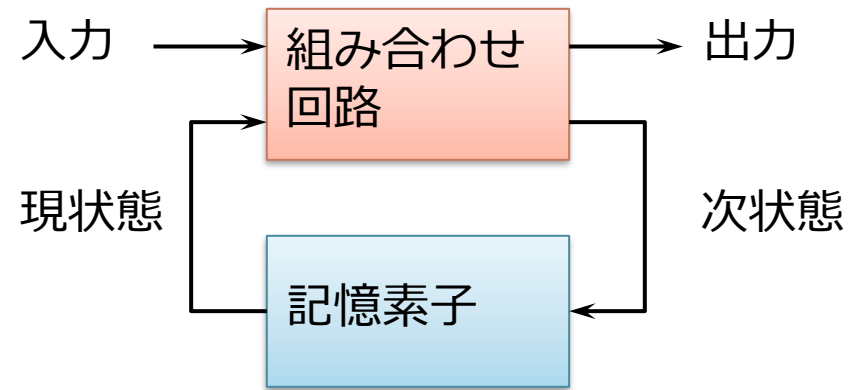
1. 組み合わせ回路

- 出力が、現在の入力のみにより決定される論理回路

2. 順序回路

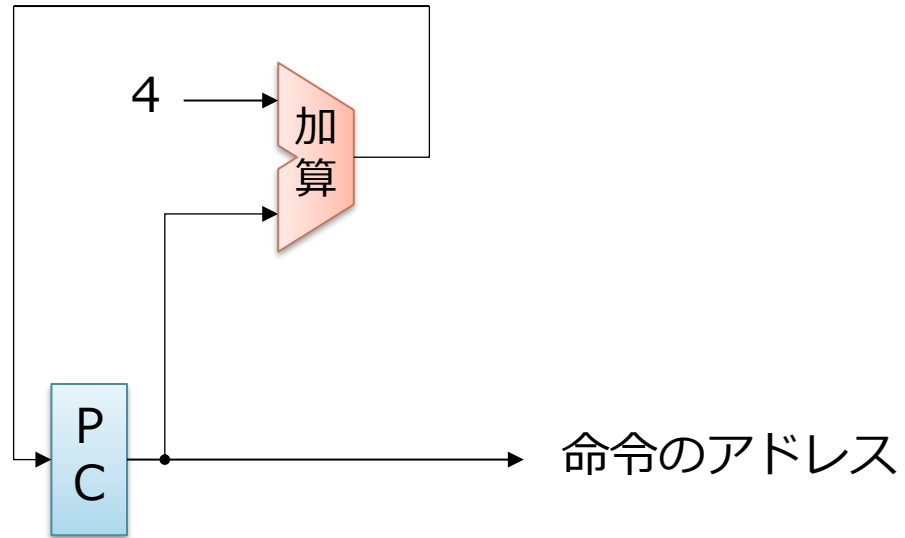
- 出力が、過去の入力（の履歴）にも依存する論理回路

順序回路



- 出力が，過去の入力（の履歴）にも依存する論理回路
 - 記憶素子と，組み合わせ回路から成る

CPU の PC 部分

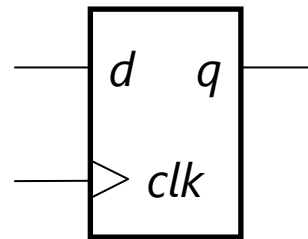


- 毎サイクル PC が加算される部分は，典型的な順序回路

記憶素子の例：D-FF (Flip Flop)

■ 入出力端子

- ◇ データ入力 : d
- ◇ データ出力 : q
- ◇ クロック入力 : clk

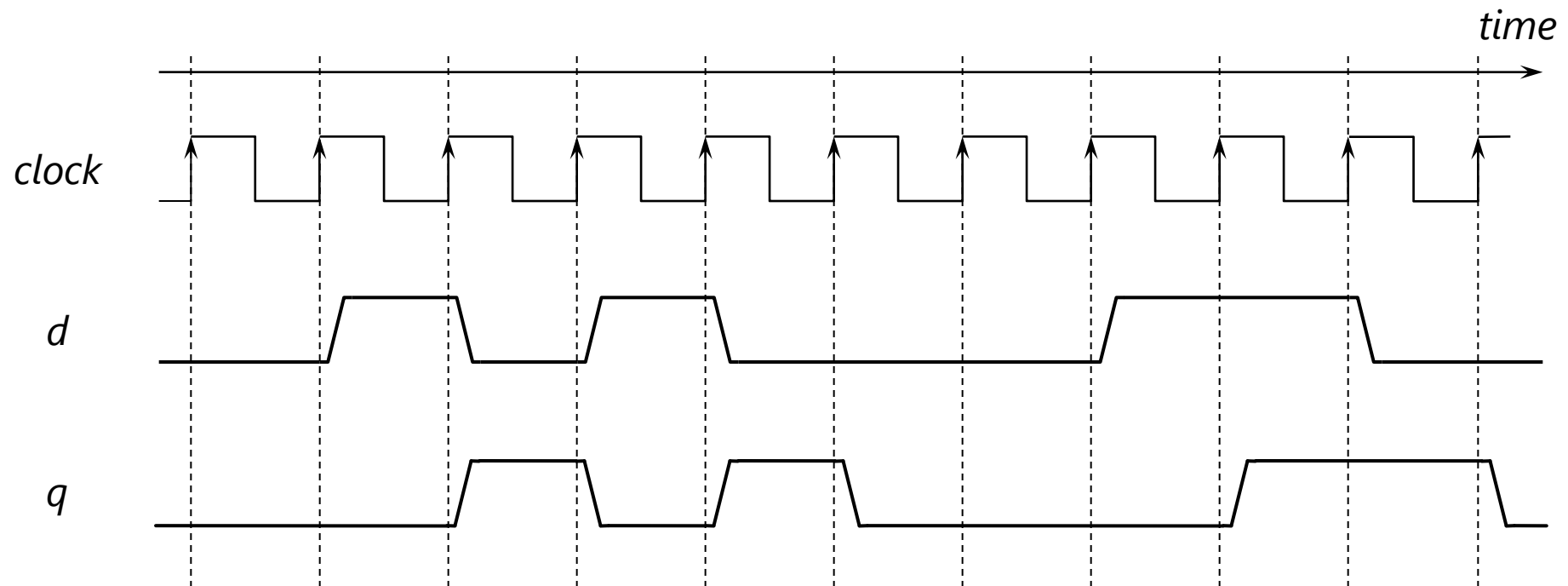


■ 働き：

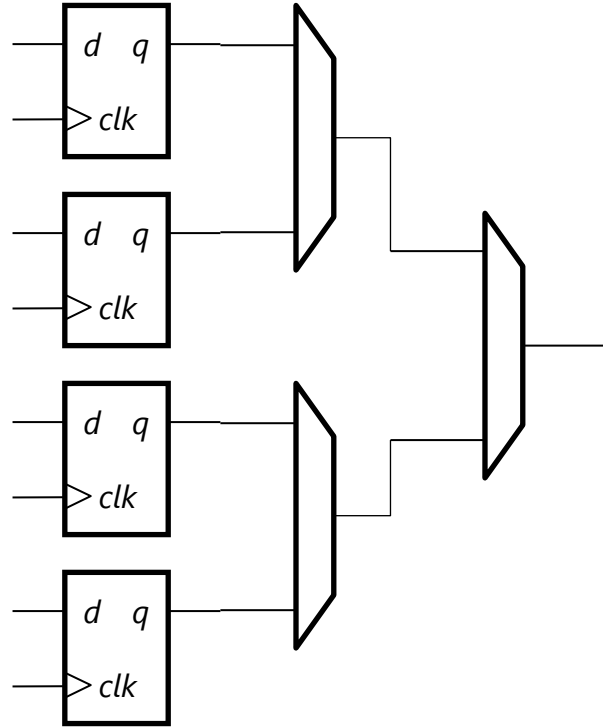
- ◇ クロックの立ち上がりのたびに, d の値がサンプリングされる
- ◇ その値が次のサイクルの間 q から出力される

D-FF の動作

d	0	1	0	1	0	0	0	1	1	0
q	0	0	1	0	1	0	0	0	1	1



メモリやレジスタ・ファイル

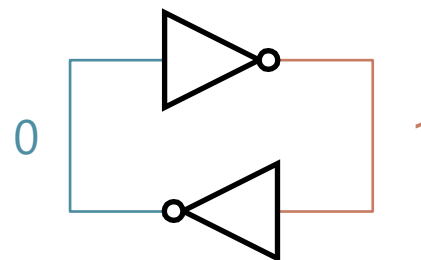
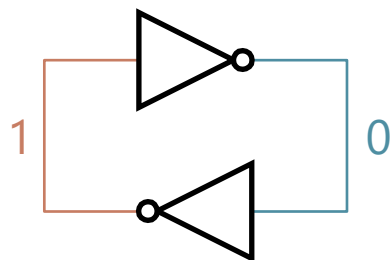


- D-FF を必要なだけ並べて，マルチプレクサで選択することで実現できる
 - 実際には，もっと最適化された回路（SRAM）が使用される事が多い
 - （後の講義で詳しく説明する予定

記憶素子の原理

■ 記憶

- 2つの NOT ゲートをループさせた回路により記憶
- 2通りの安定状態がある：1 bit を記憶

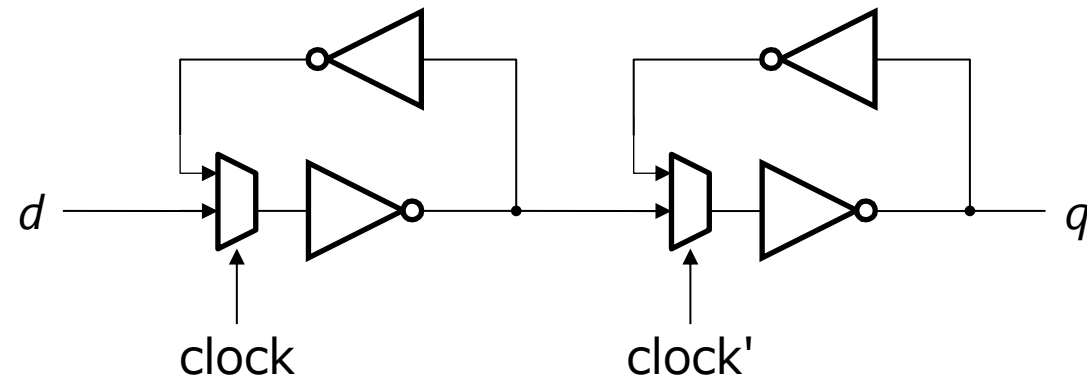


D-FFの実装

■ 構造：

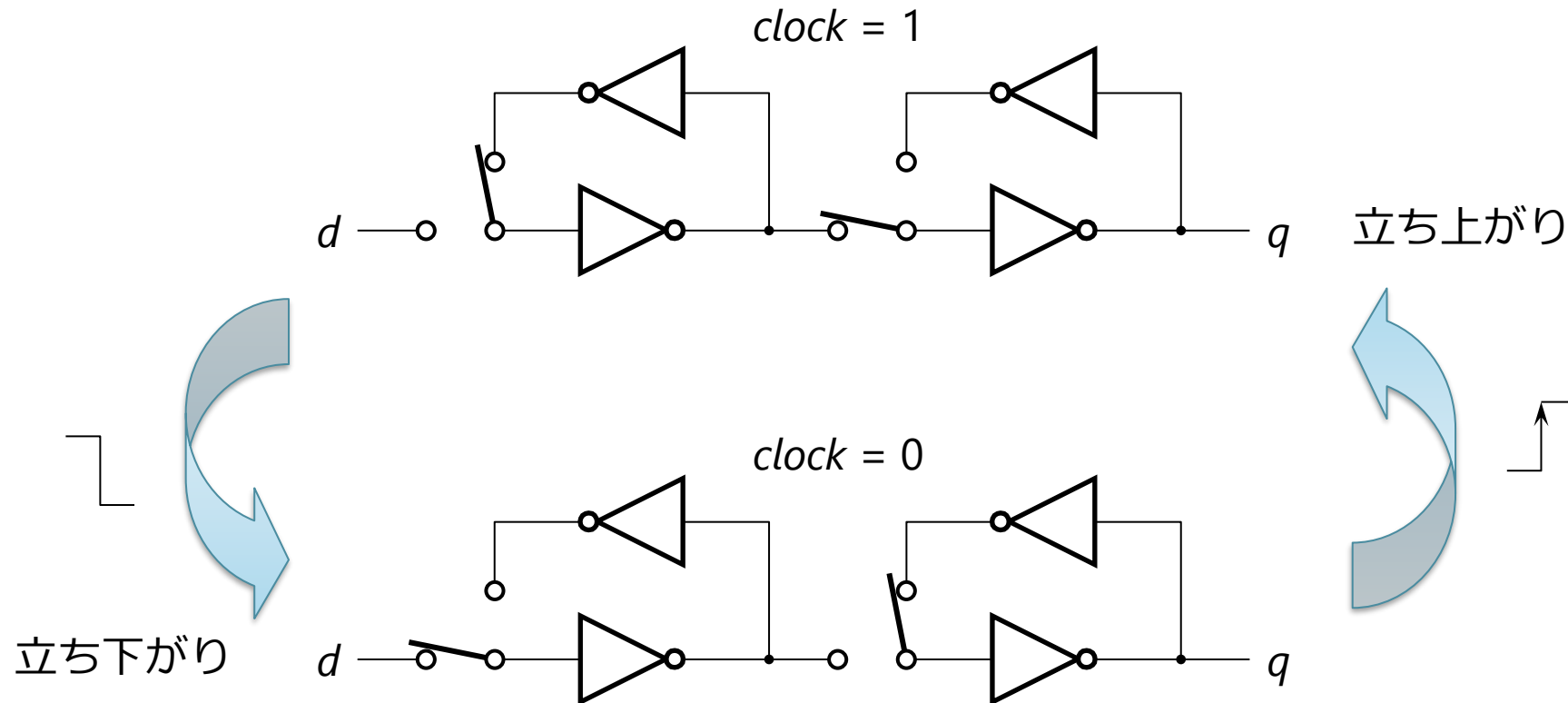
- ◇ NOT ゲートのループを二段に接続
- ◇ 各ループにはマルチプレクサが入っている

■ この構造は、クロックのエッジで記憶を更新するため



D-FF の実現例

- マルチプレクサを, 切り替えスイッチとして説明
 - ◇ クロックの立ち上がりのたびに, d の値がサンプリングされる
 - ◇ その値が次のサイクルの間 q から出力される



順序回路のまとめ

- 記憶素子も、NOT ゲートなどの論理ゲートで構成される
- 結果として、任意の組み合わせ回路/順序回路は
 - 原理的には、完全集合の要素の組み合わせに落とし込める
 - たとえば {AND, OR, NOT} を組み合わせれば、全部つくれる
- これまでに説明した CPU の要素は、全てこれでカバー可能

実際の回路生成

```
if code == 100:  
    out = rs1 xor rs2  
elif code == 110:  
    out = rs1 or rs2  
else:  
    out = rs1 and rs2
```

1. ハードウェア記述言語から，論理関数を生成：
 - ナイーブには，入力の全パターンに対して出力を記録した真理値表
 - 実際には表のサイズが爆発して，真理値表ではすぐに破綻
 - 表のサイズ = 2 の入力ビット数乗
 - さまざまな効率的な保持方法が提案されており，使用されている
 - Binary Decision Diagram (BDD)

実際の回路生成

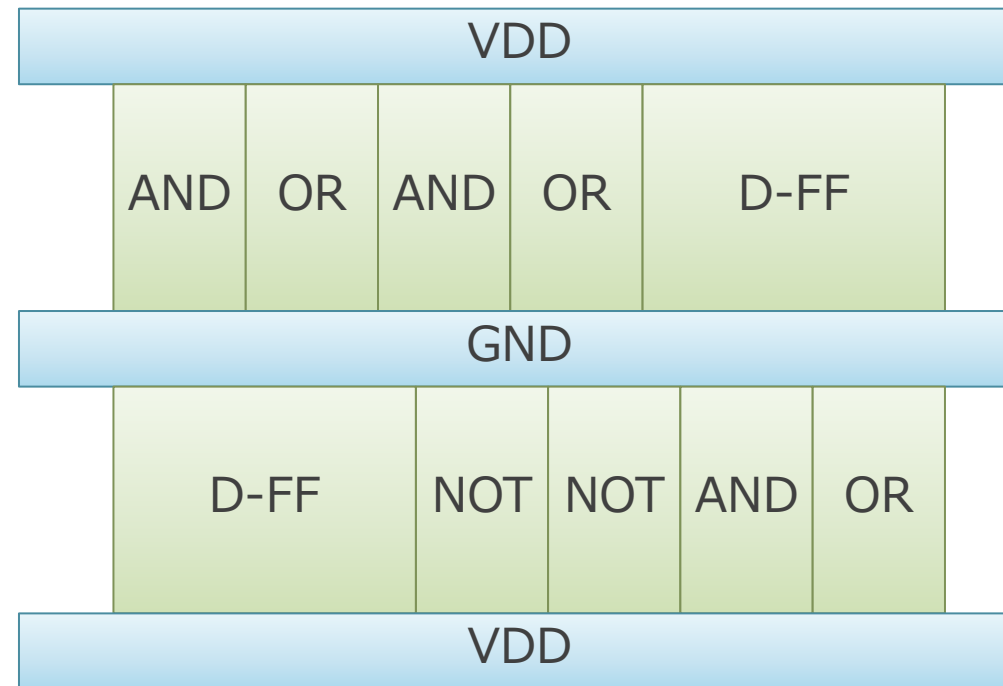
1. 論理合成：

1. ハードウェア記述言語から，論理関数を生成
2. 生成された論理関数を最適化
 - カルノー図：4入力ぐらいが限界
 - クワイン・マクラスキー法：入力数が増えると計算量が爆発
 - さまざまな最適化アルゴリズムが提案・使用されている
3. 回路のプリミティブと接続関係を生成
 - AND, OR, NOT, D-FF を始めとして，よく現れる回路の部品が用意される

2. 配置・配線：

1. 回路のプリミティブを配置・配線

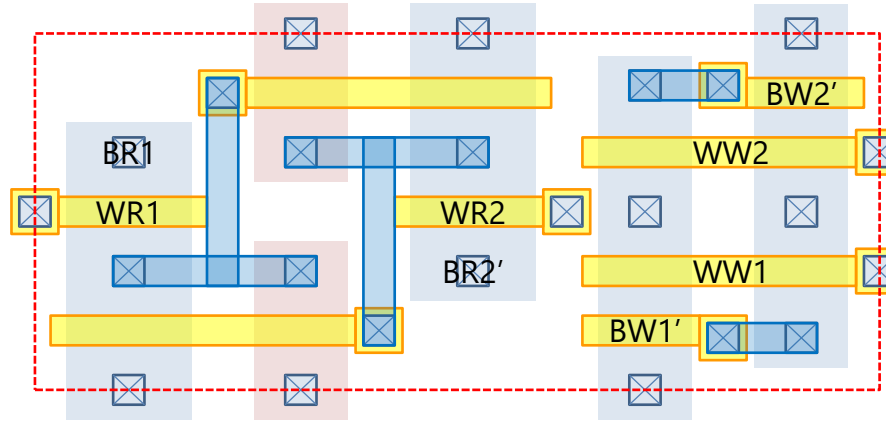
スタンダード・セル



■ スタンダード・セル：

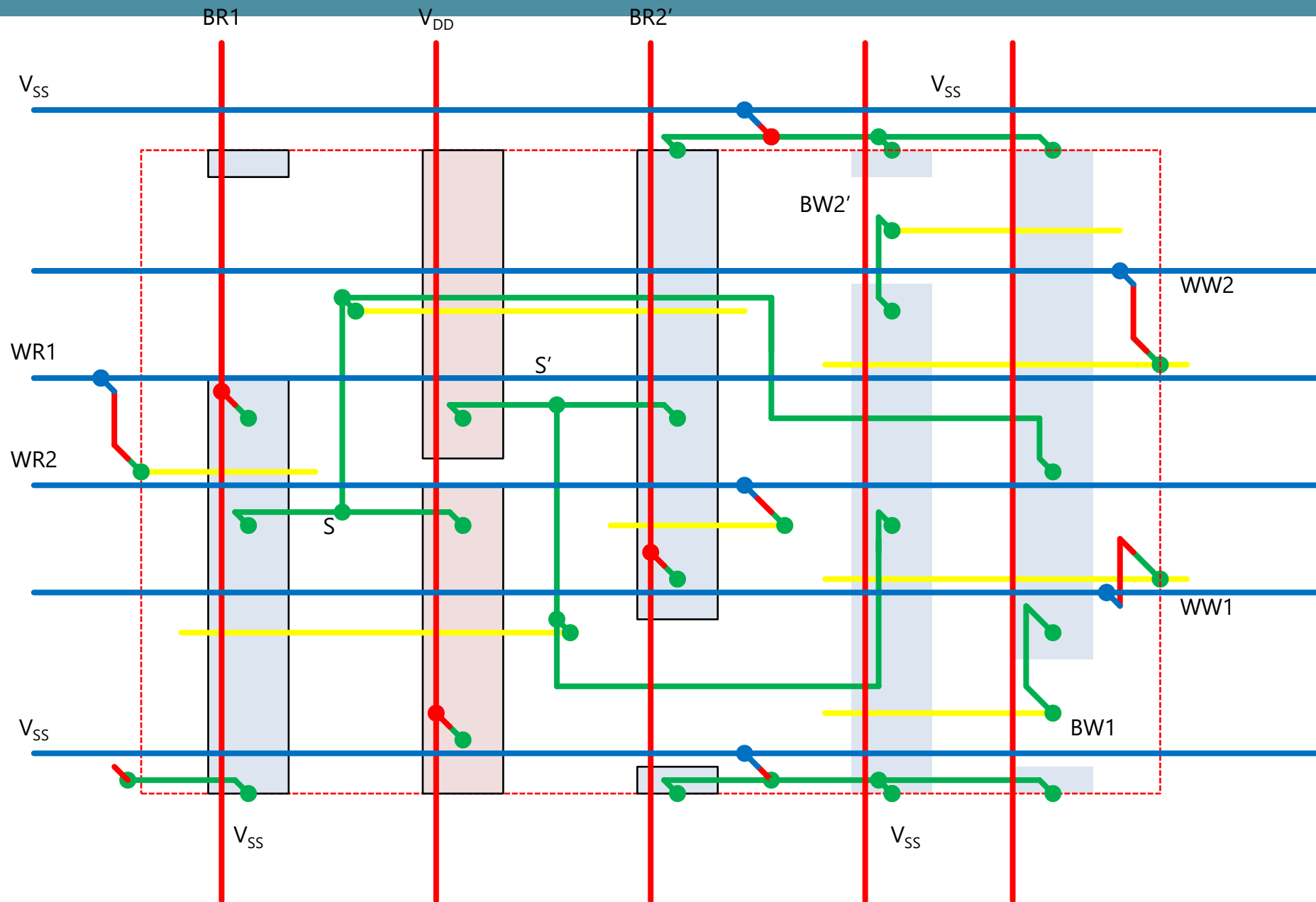
- AND や OR のようなプリミティブとなる回路「セル」を用意
 - 高さを幅を揃えておいて，簡単に並べられるように作ってある
- これらを配置して，配線を接続することにより回路を実現
 - 配置配線という

フルカスタム・レイアウト

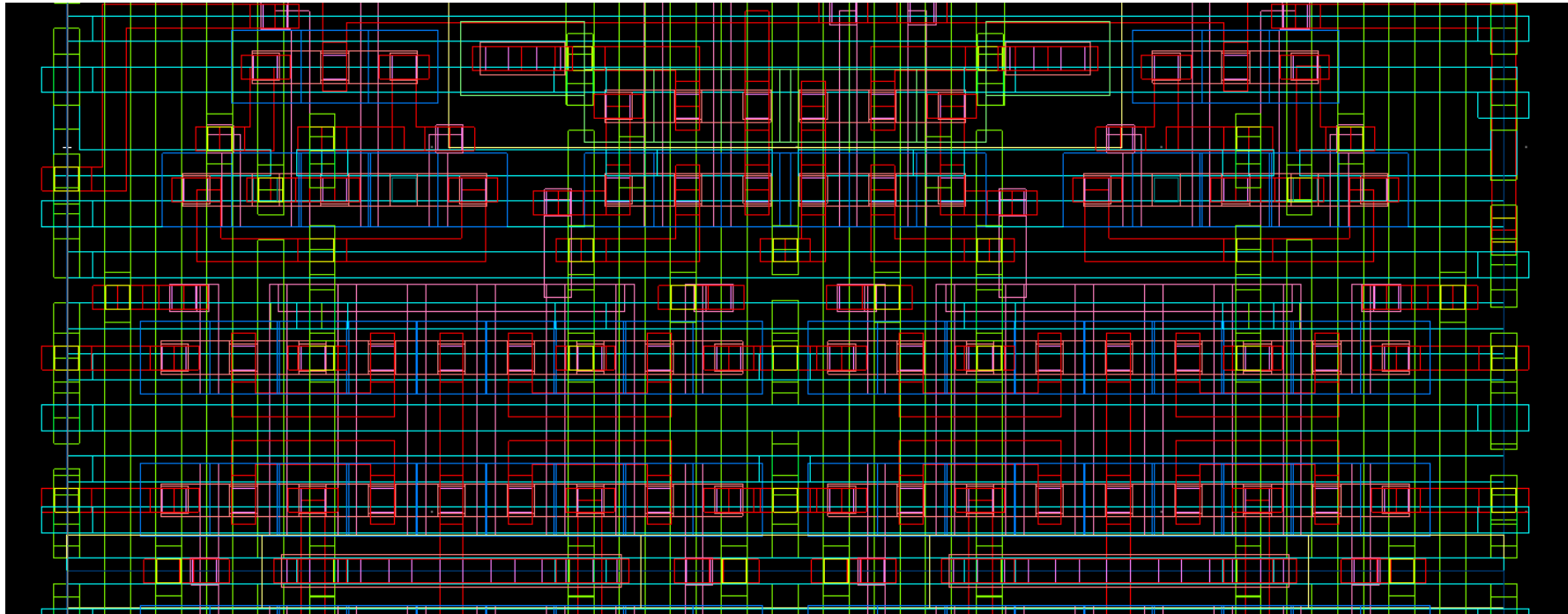


- 各スタンダード・セル（ゲート）は複数のトランジスタにより構成
- フルカスタム・レイアウト：
 - 半導体チップ上に1つ1つトランジスタをお絵かきして設計
 - 形状が特定のパターンを満たすと，トランジスタとして機能
 - 尋常じゃない手間がかかる
- 今はスタセルの中身と，本当に性能が出したい部分だけこれで作る
 - ごく一部だけアセンブリ言語でプログラムするようなもの

3次元的な構造を考えながら，設計

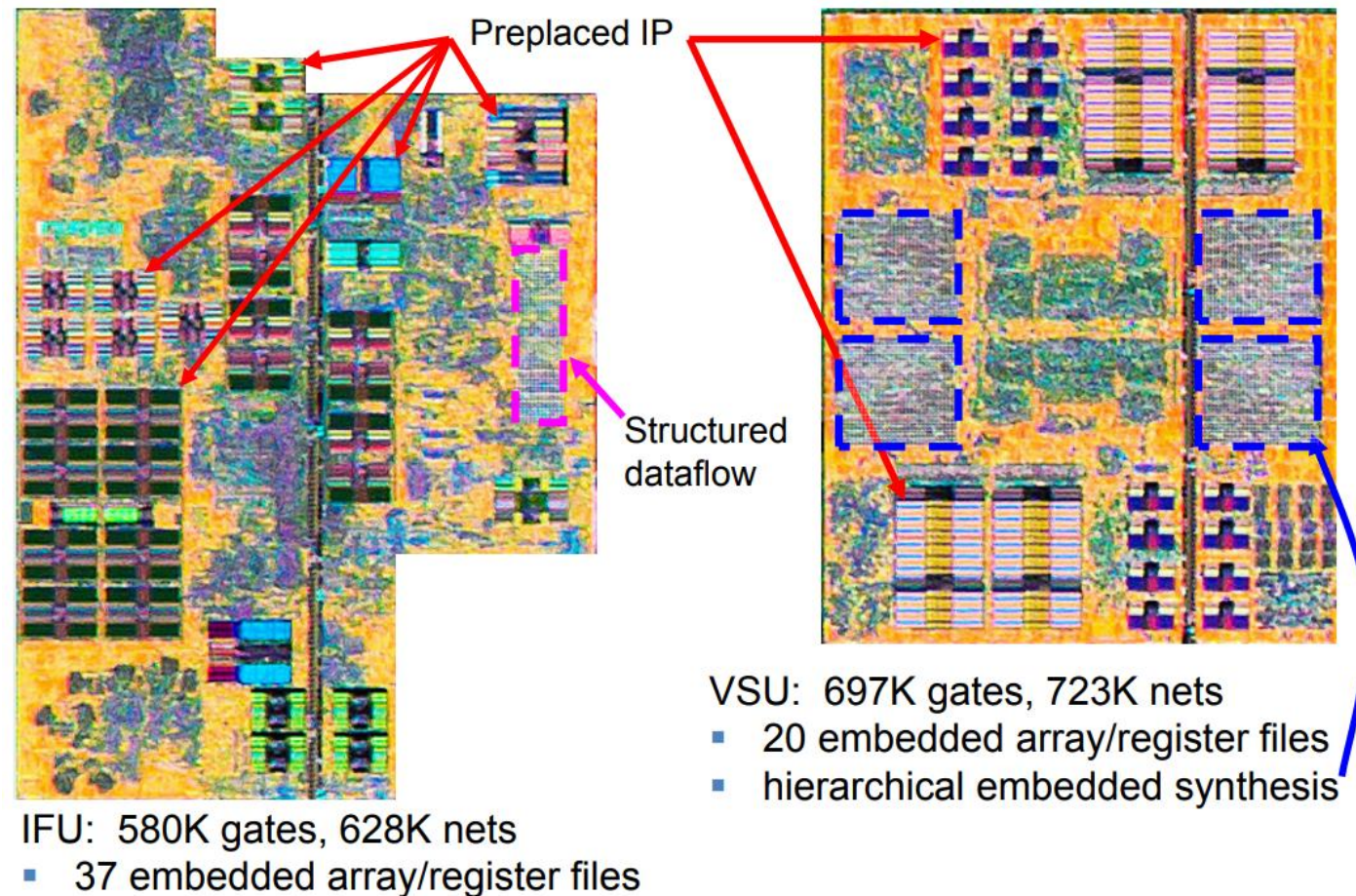


実際の設計データ



実際の例：IBM POWER8 のチップ写真

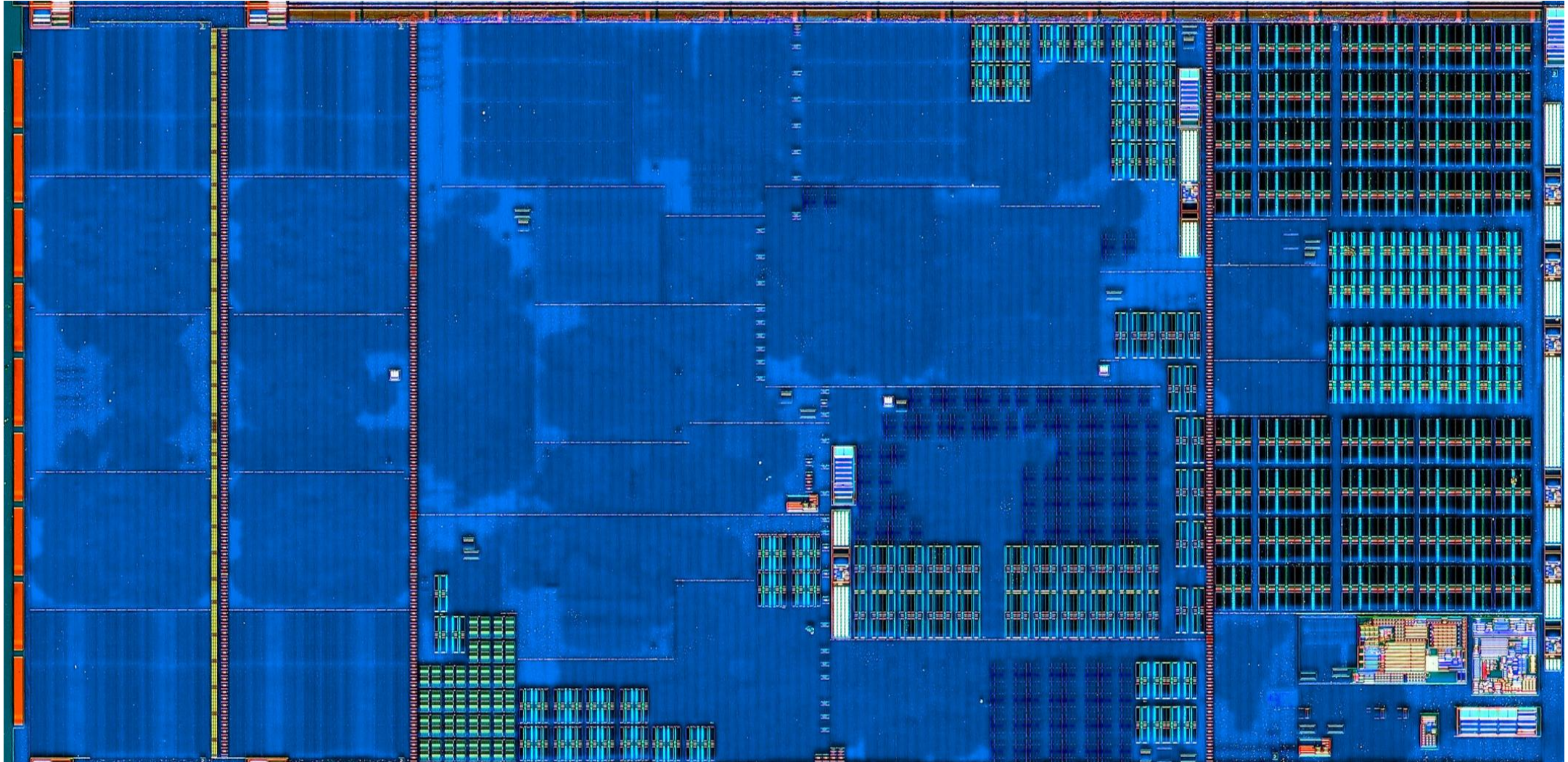
画像は 2014 IEEE International Solid-State Circuits Conference 5.1: POWER8™: A 12-Core Server-Class Processor in 22nm SOI with 7.6Tb/s Off-Chip Bandwidth より



- もわっとした模様の部分がスタンダード・セルを自動で配置した部分
- 四角いところは基本的にはメモリ
 - 同じものが規則正しく並んでいる

Zen5 のチップ写真

Fritzchens Fritz, <https://www.flickr.com/photos/130561288@N04/albums/72177720320942793/with/54045630362> より



回路の設計のまとめ

- 論理関数や記憶素子は、
すべて完全集合の要素のゲートに分解して実装出来る
- CPU の設計と製造の流れ
 1. 命令セットの仕様を考える
 2. ブロック図のレベルで設計する
 3. ハードウェア記述言語で回路を記述する
 4. 論理合成する
 5. 配置配線する
 6. 設計データを工場で焼いてもらう

論理回路の実現方法と、電力効率や遅延

現代のコンピュータの速度

- 速度は電力（とそれによって生じる熱）によってかなり律速されている
 - チップに供給できる電力はピン数などから一定
 - 排熱にも物理的な限界がある
- だいたい数百～1000 ワット程度を供給するのが限界
- 計算を行うために必要なエネルギーが、速度を決める重要な尺度となる
 - 同じ処理を行うのに必要なエネルギーが減れば、時間あたりの処理数が増える

- 回路の量と消費電力は（概ね）比例するが、なぜか？
- なぜ計算により電力が消費されるのか？
 - なぜ「情報」を処理すると電力が消費される？
 - 消費される電力の量はどのように決まる？
- これらを考える上で半導体の仕組みを大雑把に説明

- CMOS と呼ばれる方式により, 各種ゲートは実現されている
 - スタANDARD・セルの中身は CMOS
 - フルカスタム・レイアウトは, CMOS を描いている

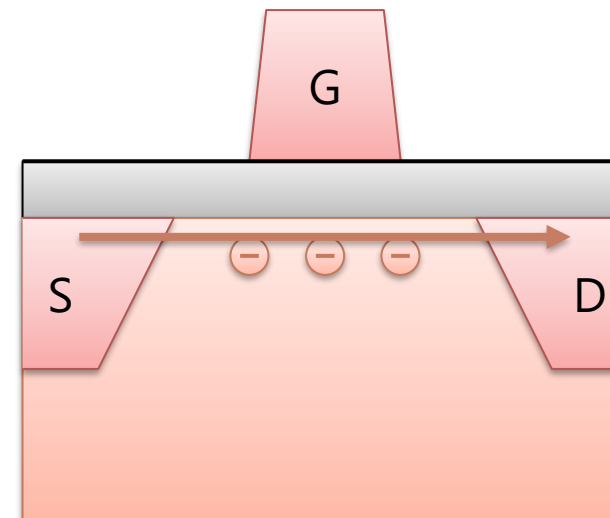
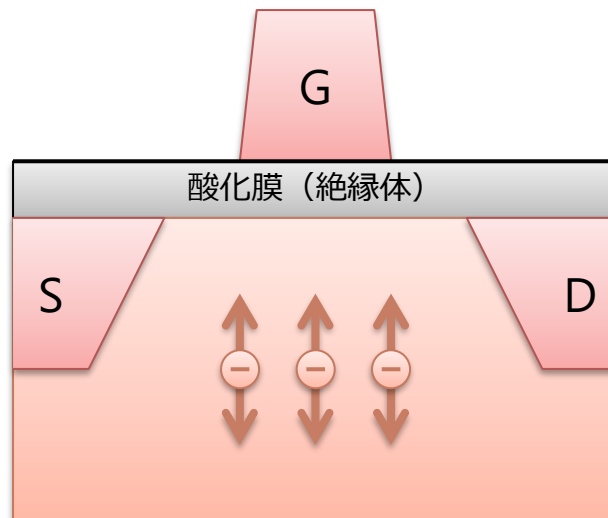
MOS FET

■ MOS: metal-oxide-semiconductor

- 上から, 金属, 酸化膜, 半導体のサンドイッチ構造
- 酸化膜を挟んで平行板コンデンサ様の構造が構成されている
 - 最近は酸化膜ではない, より誘電率が高い素材が使われる事も多い

■ 電界効果トランジスタ (FET: Field-Effect Transistor)

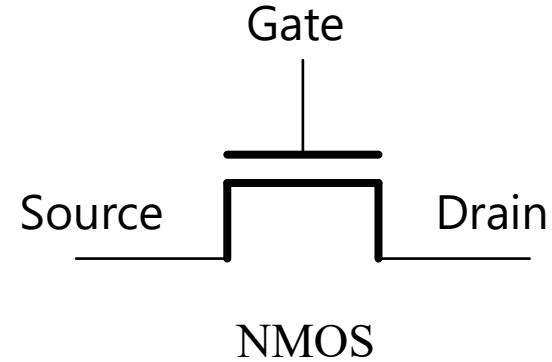
- 電界で電子を動かしてスイッチングする (NMOS の場合)
 1. G に電圧をかけてコンデンサに充電
 2. 電子の層 (チャネル) 酸化膜下にできて, S と D が繋がり ON に



MOS には NMOS と PMOS の2つがある

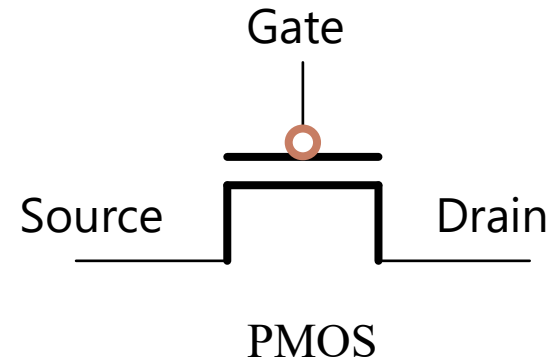
■ Negative (N)

- 電子がキャリア
- 低電位しか通せない
- 接地側に配置



■ Positive (P)

- 正孔 (hole) がキャリア
- 高電位しか通せない
- 電源側に配置
- Gate に○がついている

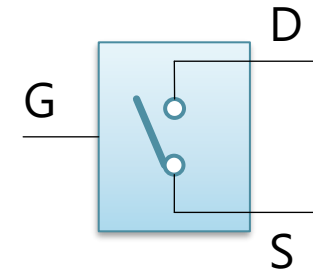
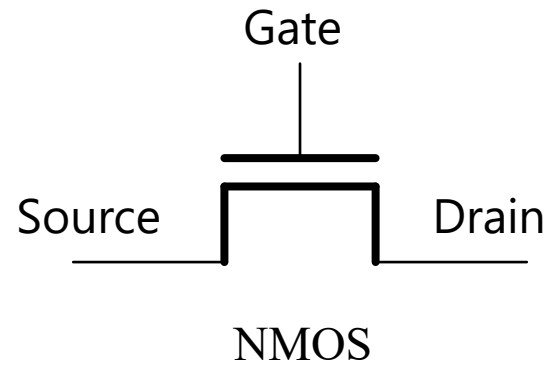


■ この2つを相補的に組み合わせるので CMOS という

入力で ON/OFF が切り替わるスイッチと考える事ができる

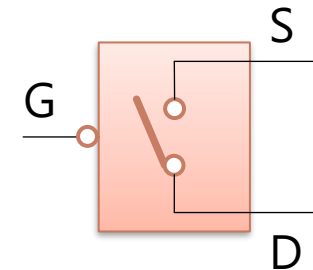
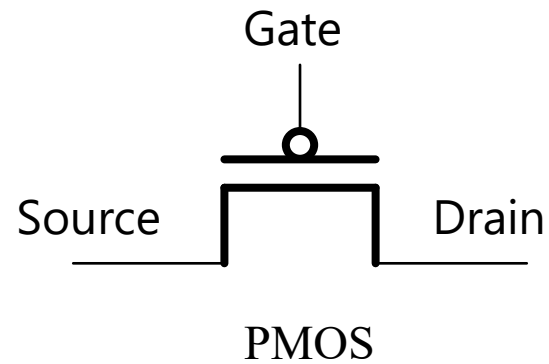
■ Negative

- Gate (G) を 1 にすると ON になる



■ Positive

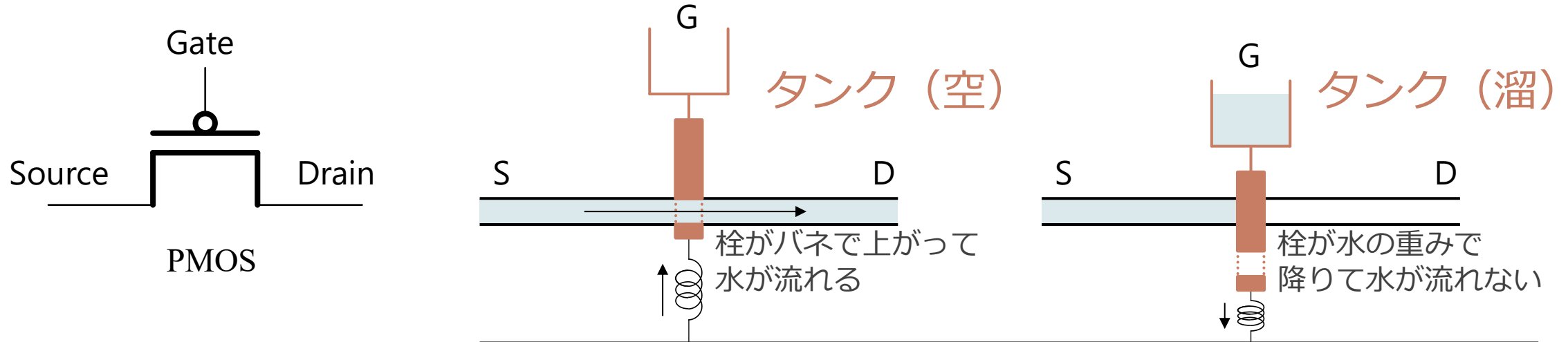
- Gate (G) を 0 にすると ON になる



だがしかし

- いきなり MOS とか言われても、何もわからないのではないかと
 - 消費電力はコンデンサへの充放電により説明されるが、コンデンサが何かとか憶えてない（or 知らない）のでは
- もうちょっと直感的な例を使ってイメージを掴みたい
 - ここでは水の流れに例えて説明
 - 技術的に正確ではないが、かなりイメージは一致する
 - 一次近似としては良くあっているはず
 - 消費電力のイメージもかなり伝えやすい
- 回路の量（大きさ）と、電力の関係のイメージを掴んでほしい

水の流れて例えることにする



■ 水流タンクシステム：

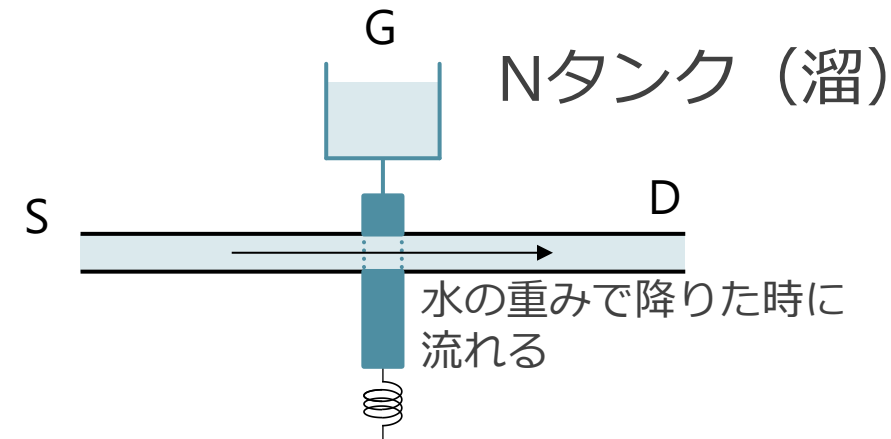
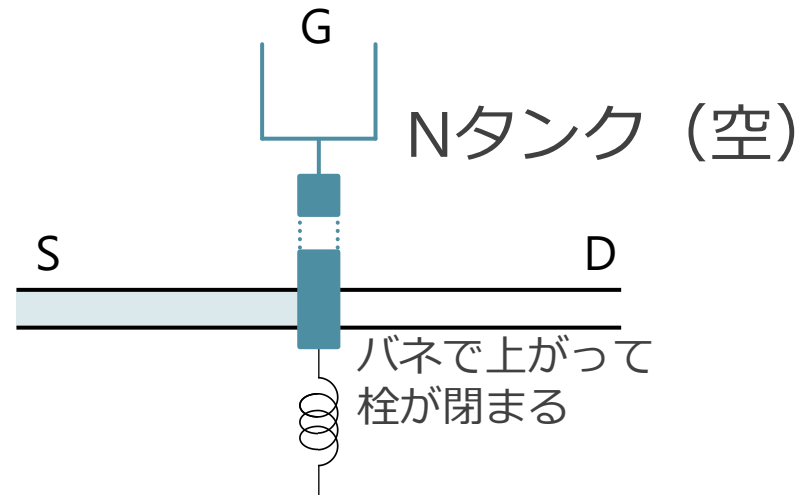
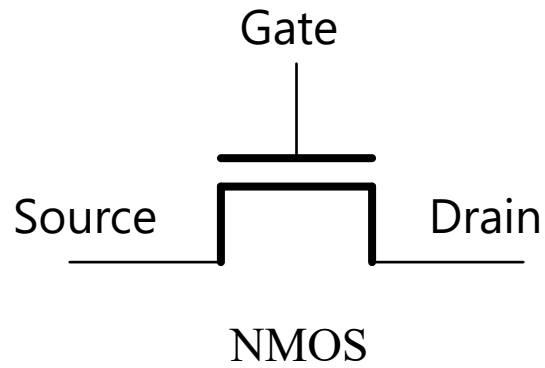
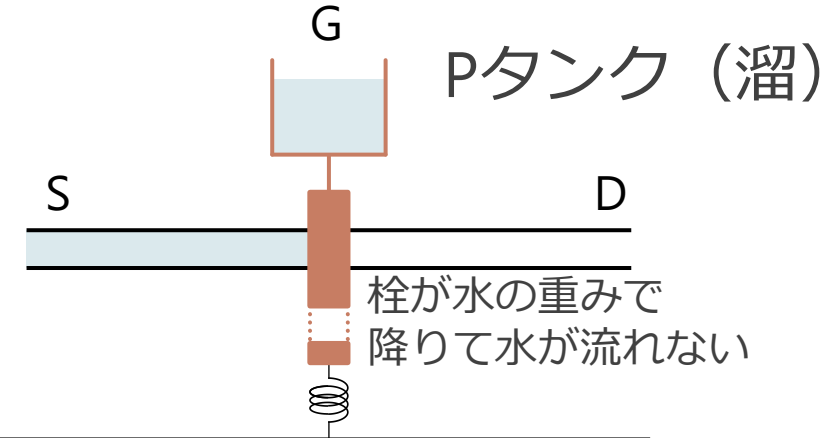
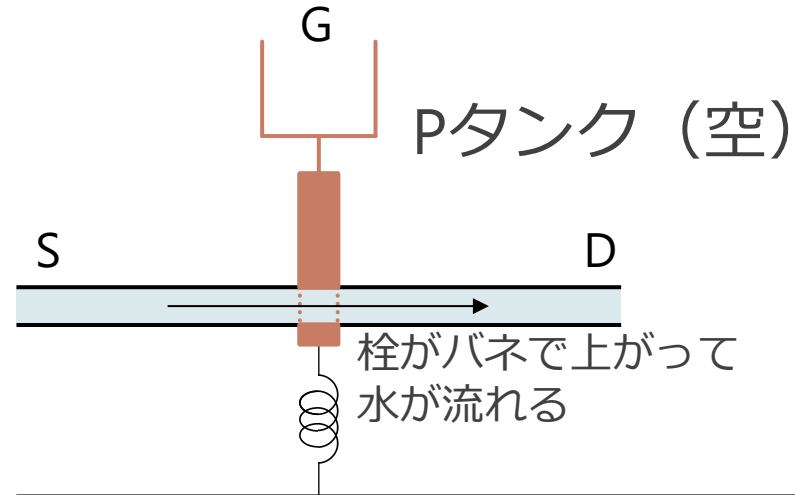
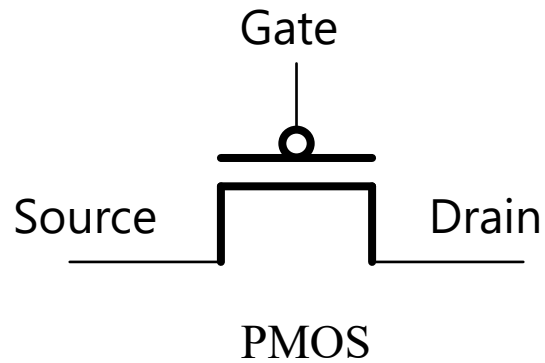
- 水が流れるパイプとタンク+栓を考える
- タンクに水がたまっているかどうかで連動して動く栓を持つ
 - 栓には水が通れる部分があって、水流を制御できる

■ PMOS と同様の挙動：

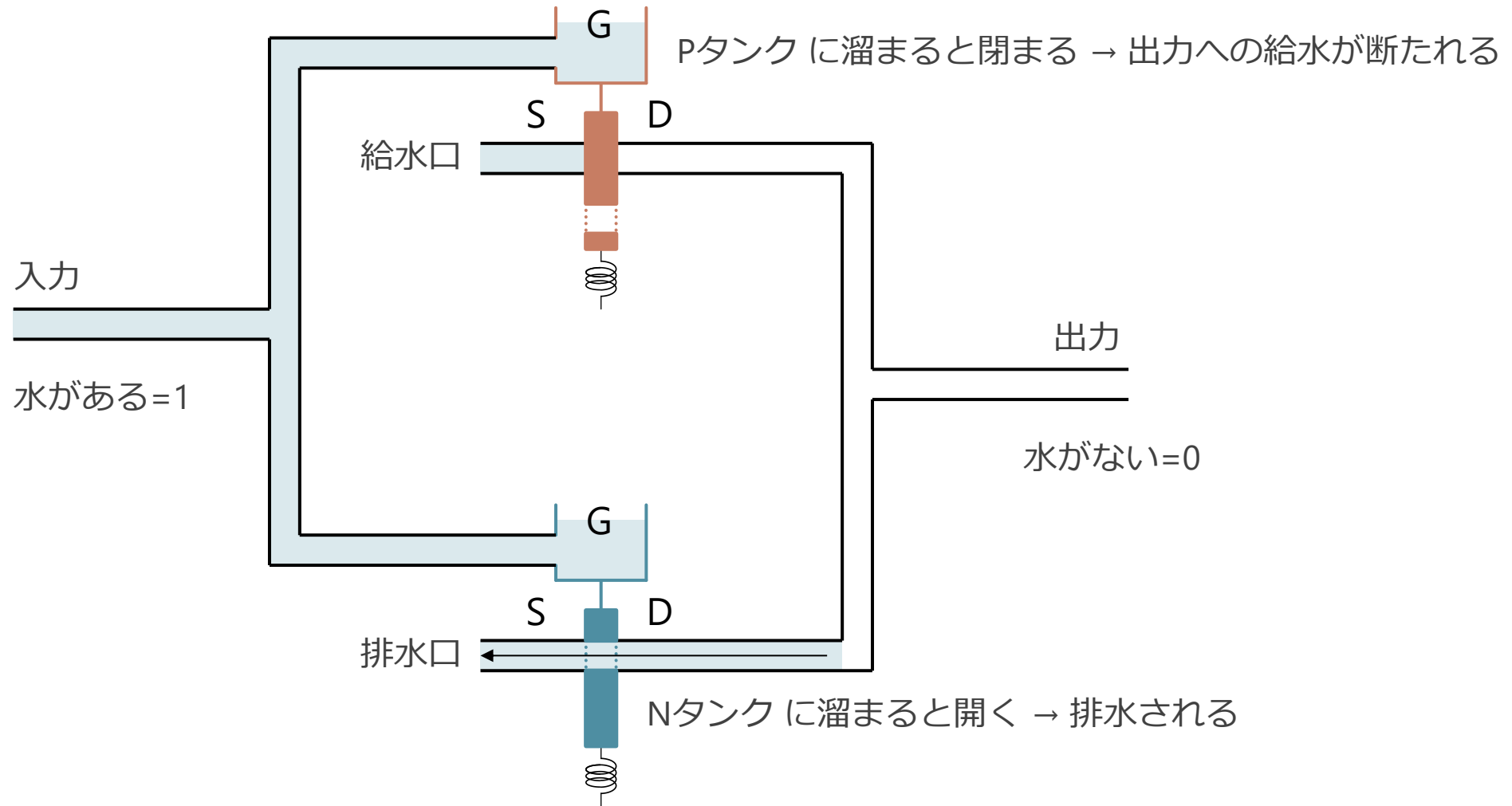
- 「タンクが空だと、S から D に水が流れる」
- 「電荷が Gate から抜けて空だと、S から D に電荷が流れる」

タンクは2種類ある (PMOS と NMOS に相当)

タンクに水が溜まっている時に、流れるか遮断されるかが逆になっている

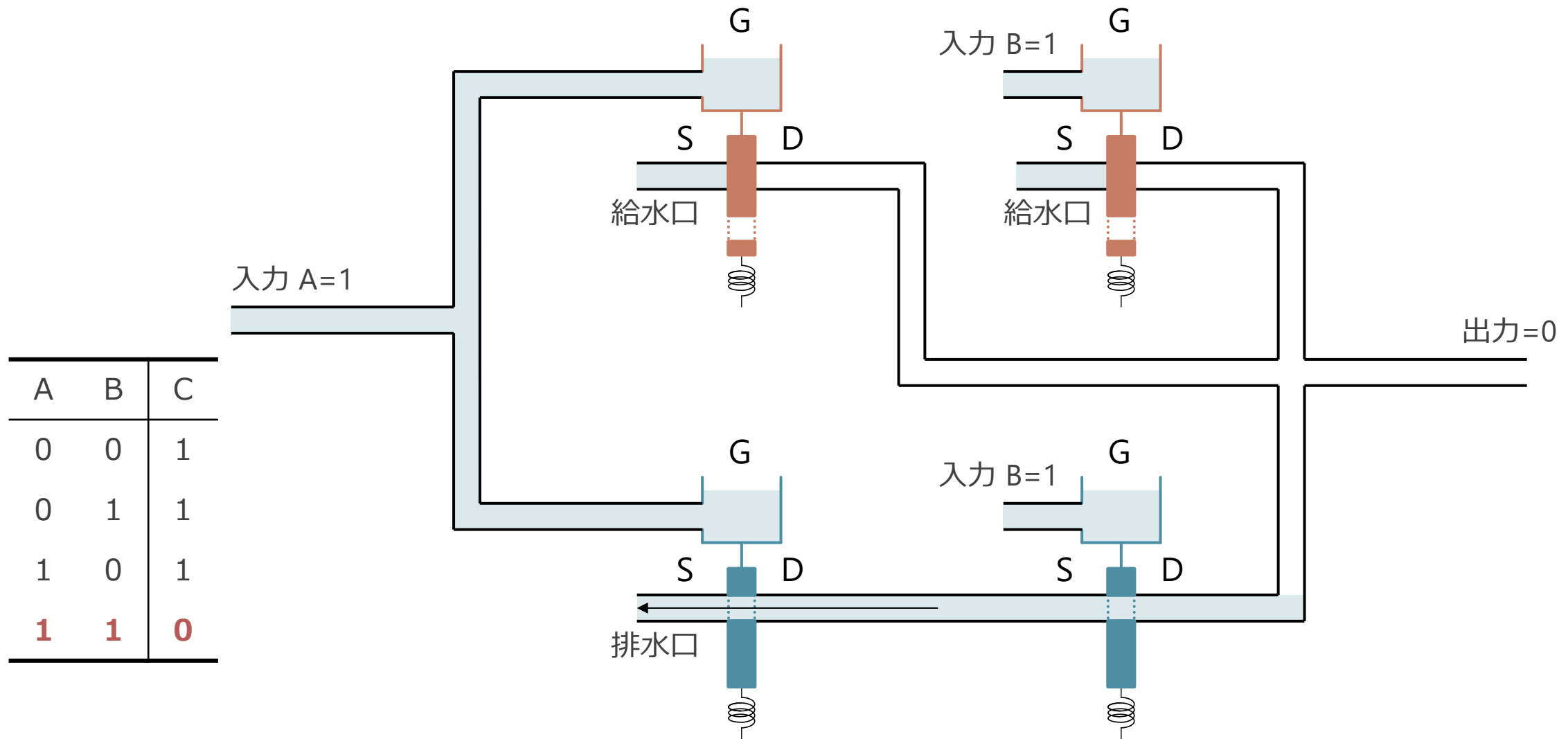


P タンクと Nタンクで NOT ゲートが作れる



- 給水口には常に一定の圧力で水が供給されている (=電源)
- 入力側の水の有無により, 出力側の水の有無が反転 → NOT ゲートの動作

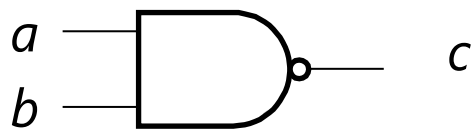
上側を並列繋ぎ、下側を直列繋ぎにすると NAND になる



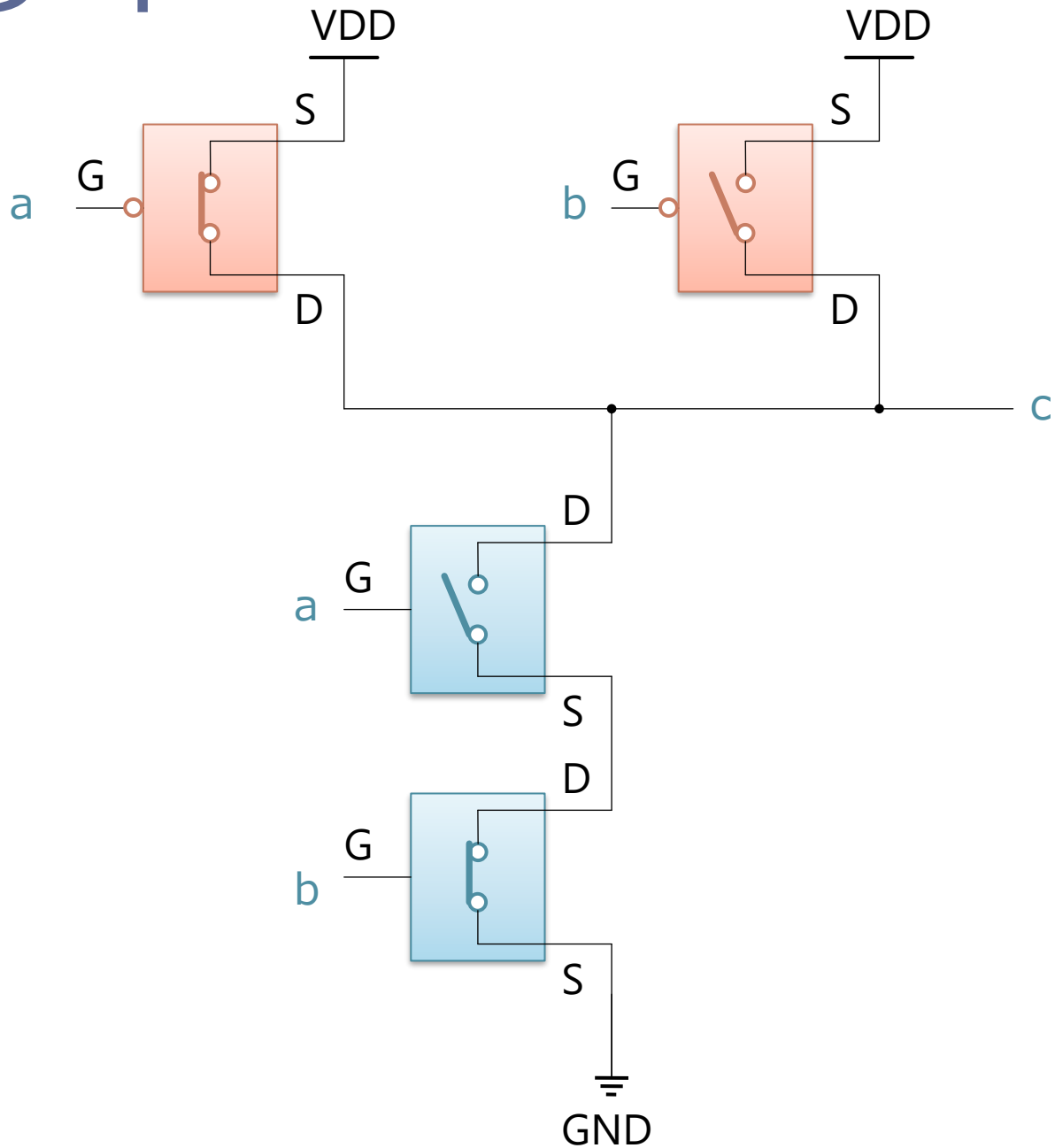
- 下側の2つのNタンクは、直列に繋がってるので双方溜まった時のみ排水が有効 → NAND

スイッチで書いたNAND ゲート

- 右のように接続すると NAND ゲートが作れる



a	b	c
0	0	1
0	1	1
1	0	1
1	1	0



スイッチ的なものがあると NAND が作れる → コンピュータが作れる

- ここまで：
 - スイッチ的なものがあると NOT と NAND が作れる
- なんらかの入力に従って ON/OFF できるスイッチがあれば,
それでコンピュータが作れる

スイッチからコンピュータを作る

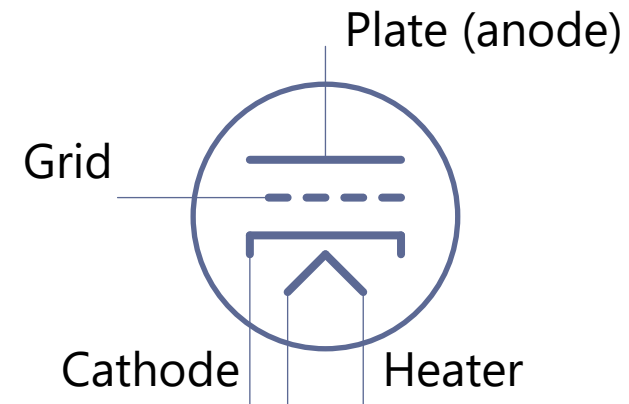
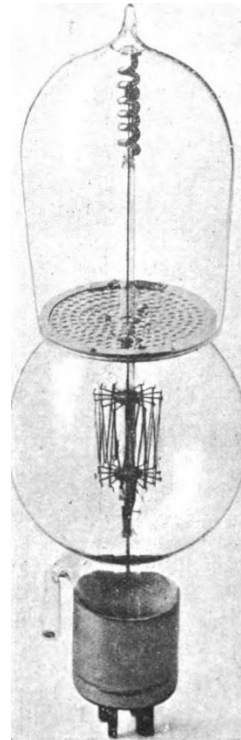
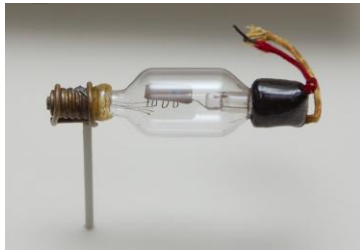
- スイッチから NOT, AND, OR を作る手順
 1. スイッチを組み合わせて NOT と NAND を作る
 2. NOT と NAND を組み合わせて AND と OR を作る
- NOT, AND, OR からコンピュータを作る手順
 1. NOT, AND, OR を使うと任意の論理回路が作れる
 2. 任意の論理回路を使うと CPU の部品が作れる
 - PC, レジスタ, 演算器, メモリ...
 3. それらを組み合わせるとコンピュータができる

つまり,

- なんらかの入力に従って ON/OFF できるスイッチがあれば,
それでコンピュータが作れる
 - 具体的な回路の組み方は, 回路の種別ごとに違う事もある
 - NAND さえ出来ればこっちのもの

真空管（三極管）

- 電球にカソードとグリッド，プレートなどの電極を追加したもの



画像は <https://en.wikipedia.org/wiki/Triode> より
De Forest Audion tube と Lieben-Reisz tube
1900 年過ぎの発明

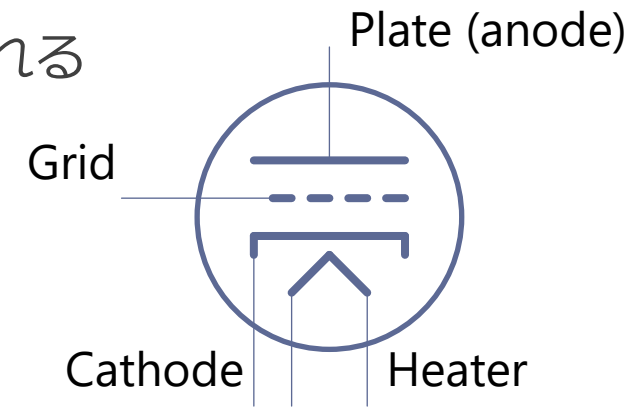
真空管の動作

■ 動作：

1. Plate に正の電圧を, Cathode に負の電圧をかける
2. Heater (電球のフィラメント) に電源を接続
3. 暖められた Cathode が電子を放出
4. Plate に電子が吸い寄せられる
 - 電子が移動する = それらの間に電流が流れる

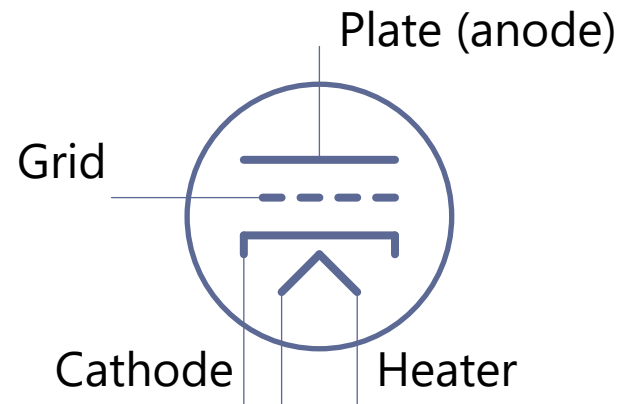
■ スイッチング

- なにもしなければ Grid は網なので電子はすき間を通過
- Grid に負の電圧をかける
- 負同士ではじきあうので, 電子が上にいけなくなり電流が OFF

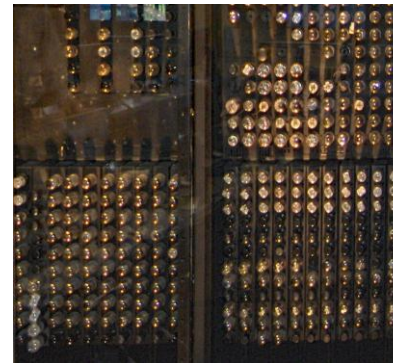
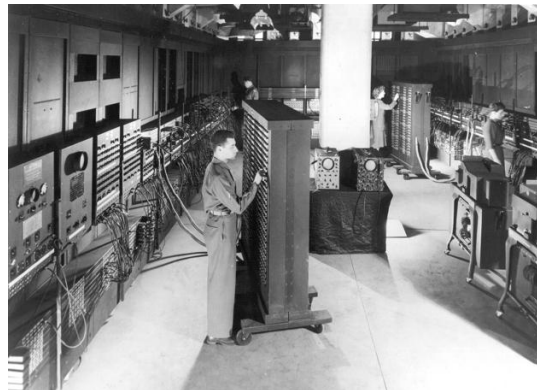


真空管

- Grid にかける電圧で, Cathode と Plate を ON/OFF できる
 - つまり, これでもコンピュータが作れる



- ENIAC (1945)
 - 世界で最初の汎用コンピュータ
 - 18,000 個の真空管で出来ている



レッドストーン回路

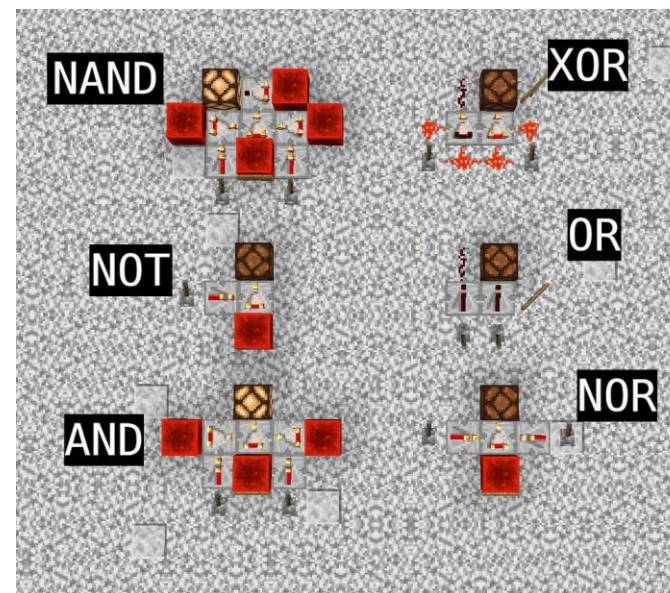
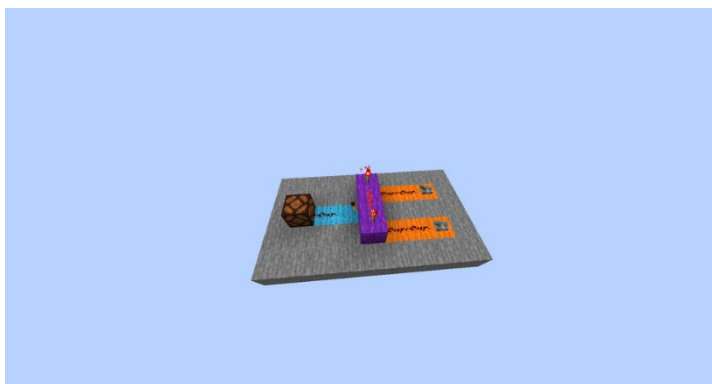
- Minecraft というゲームの中で作られている回路
 - <https://www.minecraft.net/ja-jp>
 - もともとは箱庭でブロックをおいて色々つくって遊ぶゲーム



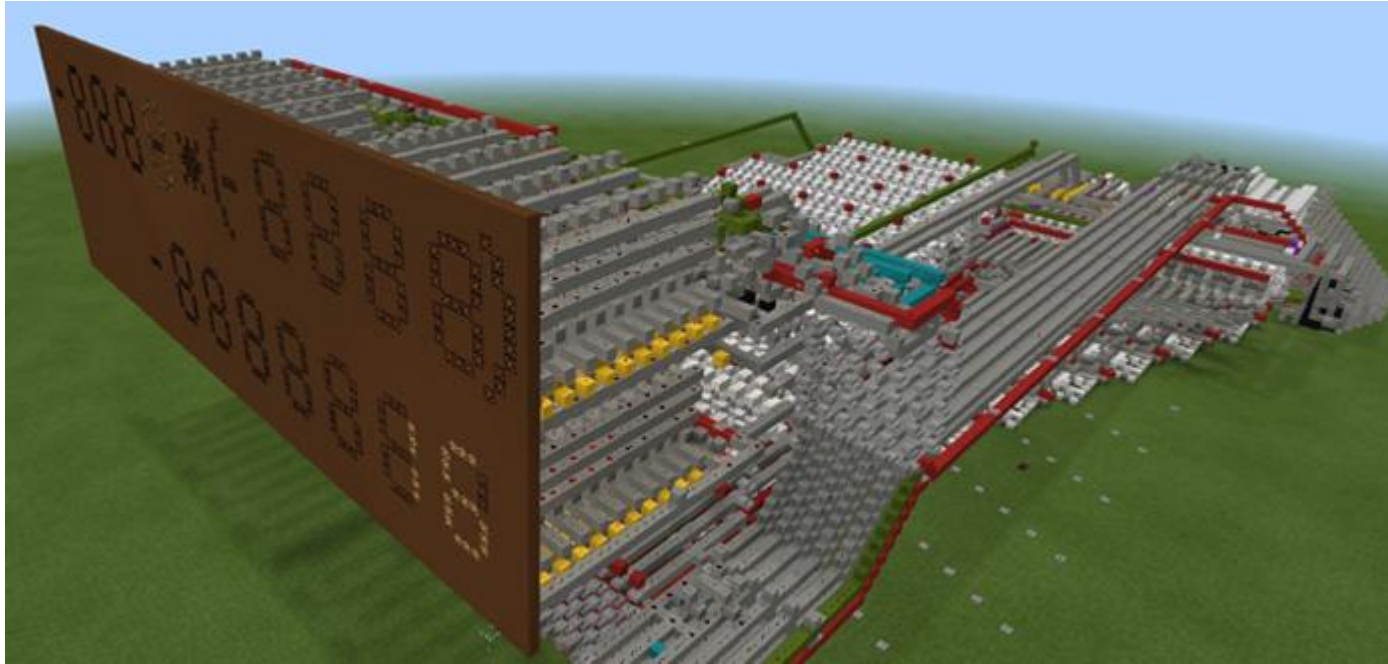
レッドストーン回路

- 「レッドストーン」というゲーム内の要素を利用
 - 有効と無効の状態を持つ
 - スイッチと扉を繋げて、遠くから開けたり閉じたりとかする
- これも入力で出力を制御出来ていることに誰かが気づいた

AND ゲート
右のスイッチを2つとも入れると
左側が赤くなる



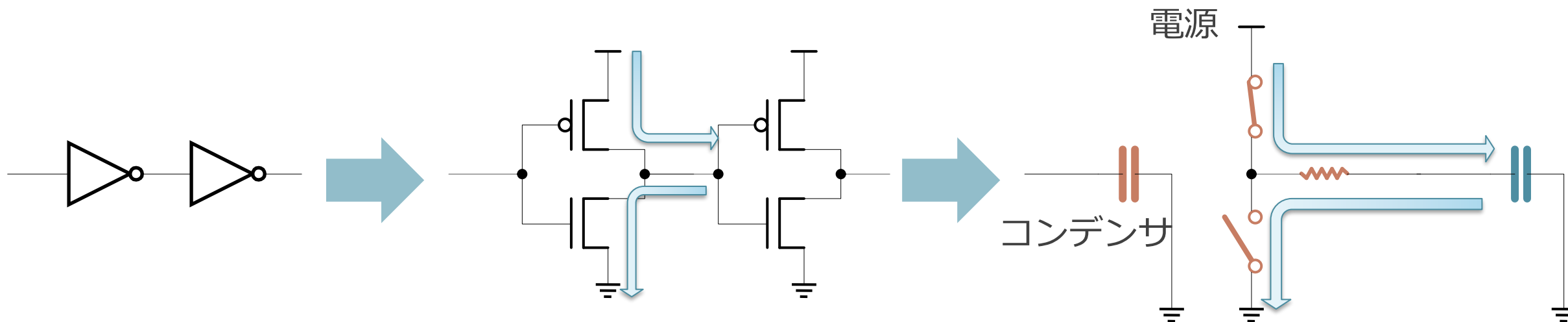
入力で出力を制御出来ればコンピュータが作れる



画像は <https://mcpedl.com/raffis-calculator-2-map/> より
四則演算ができる計算機・・・らしい

CMOS と水流システムの消費エネルギー

CMOS 回路におけるスイッチングの消費エネルギー

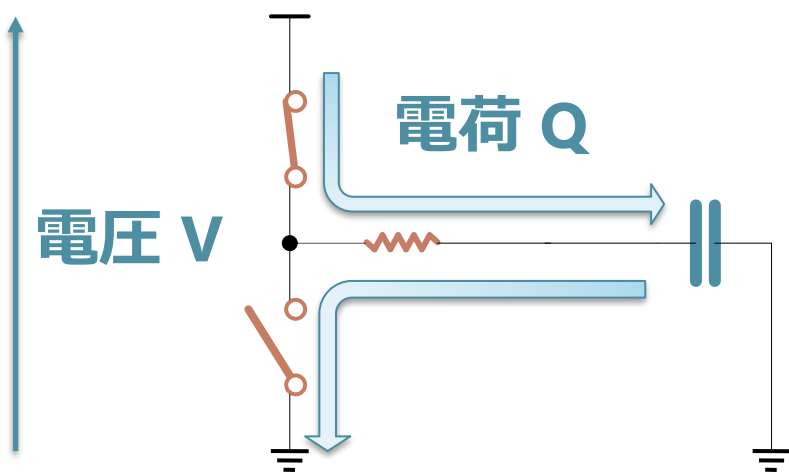


- CMOS 回路は、コンデンサに連動したスイッチとみなせる
(MOS 構造のゲートのところがコンデンサになってる)
 - コンデンサが充電状態：下のスイッチがON
 - コンデンサが放電状態：上のスイッチがON
- 主に計算で消費されるエネルギー：
 - スイッチを ON/OFF するためのコンデンサへの充放電で消費

これも水流で考える

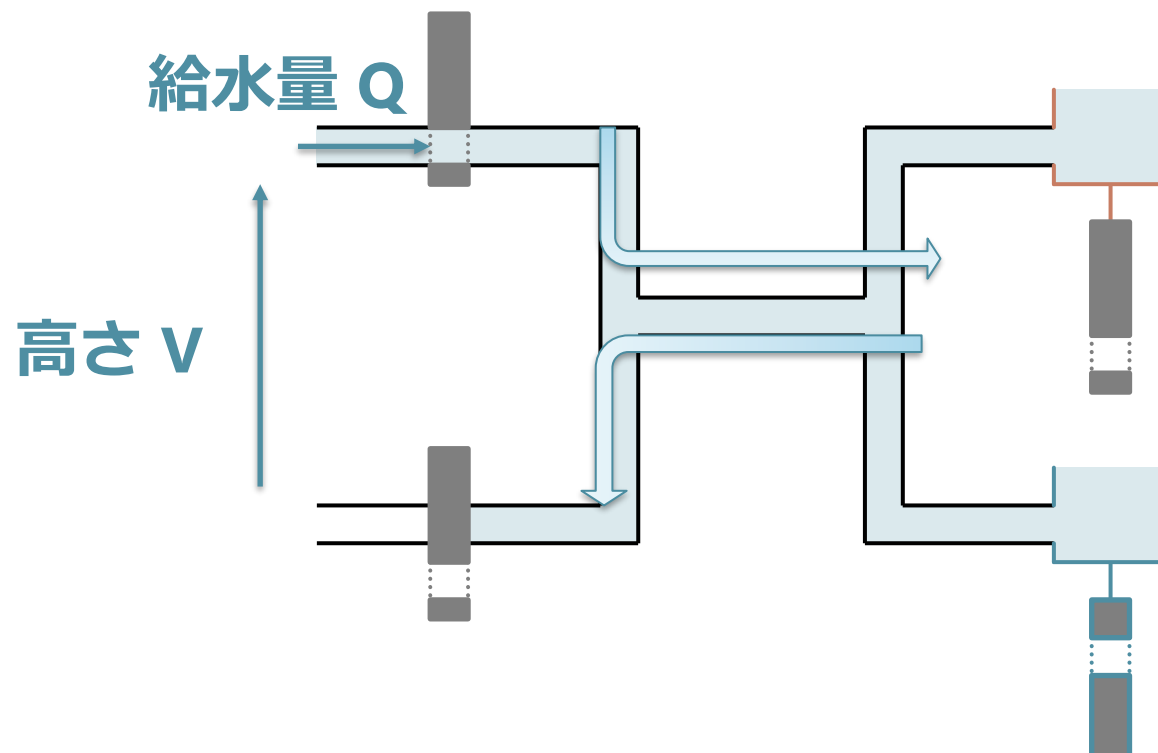
■ 電流を流して失われるエネルギー

- コンデンサに溜まる
電荷の量 : Q
- 電圧 : V
- 消費エネルギー : $Q \times V$

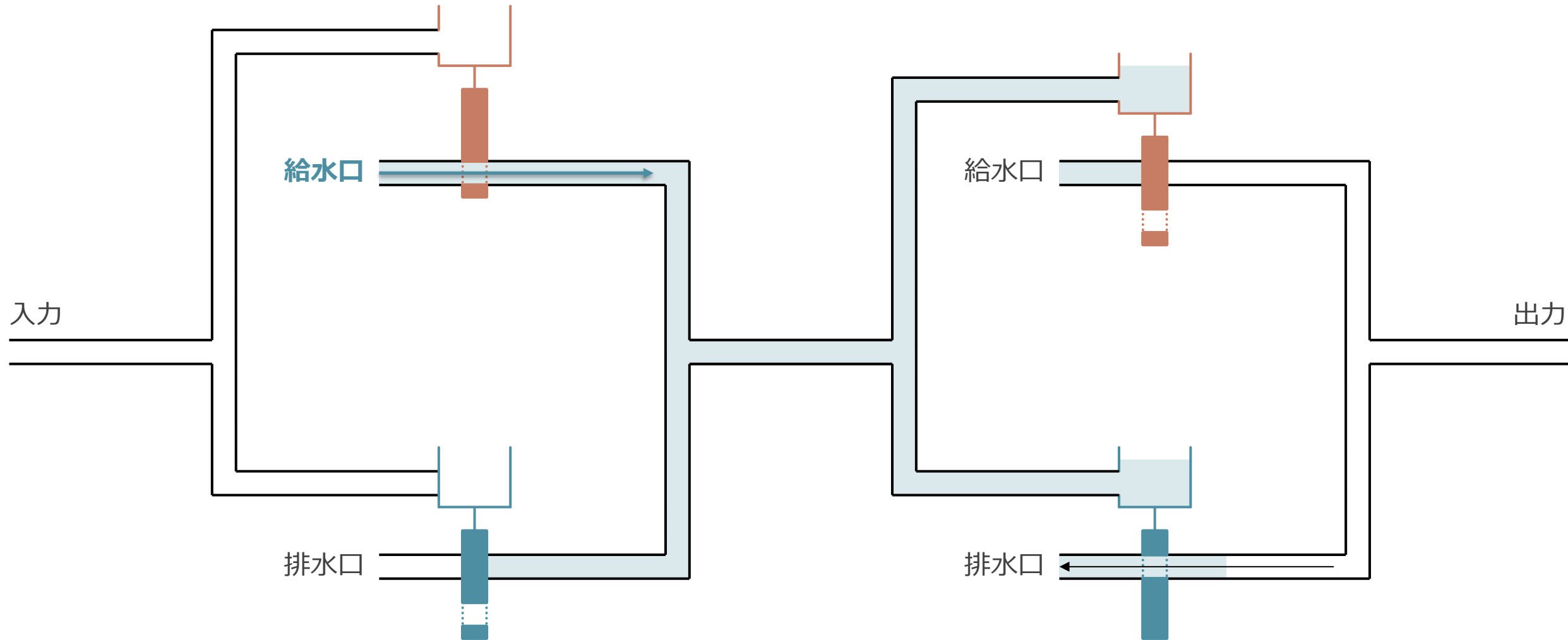


■ 水を高いところから流すとその分位置エネルギーが失われる

- 水の総量 : Q
- 排水口から給水口高さ : V
- エネルギー : $Q \times V$

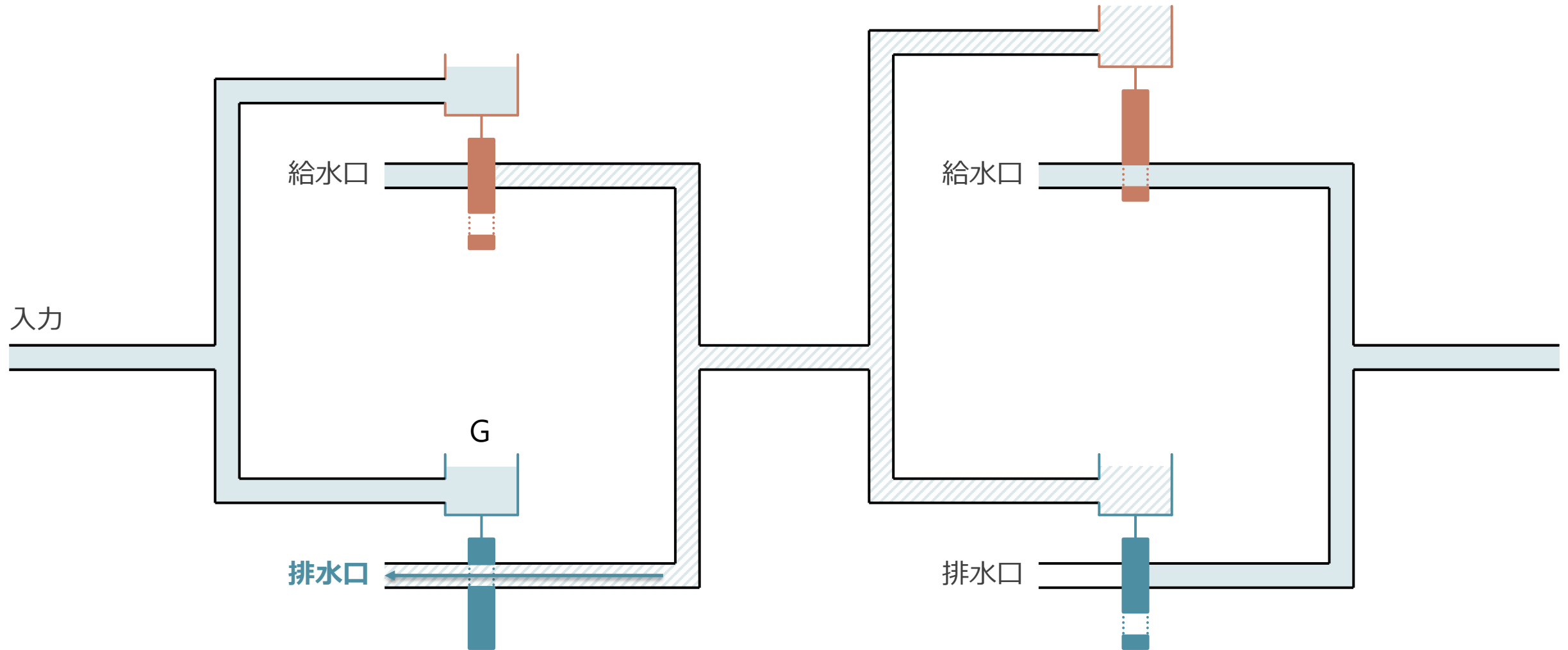


NOT ゲートを 2 段重ねた例：入力が 0 の場合



- 左上の給水口から，右側 2 つのタンクに給水される

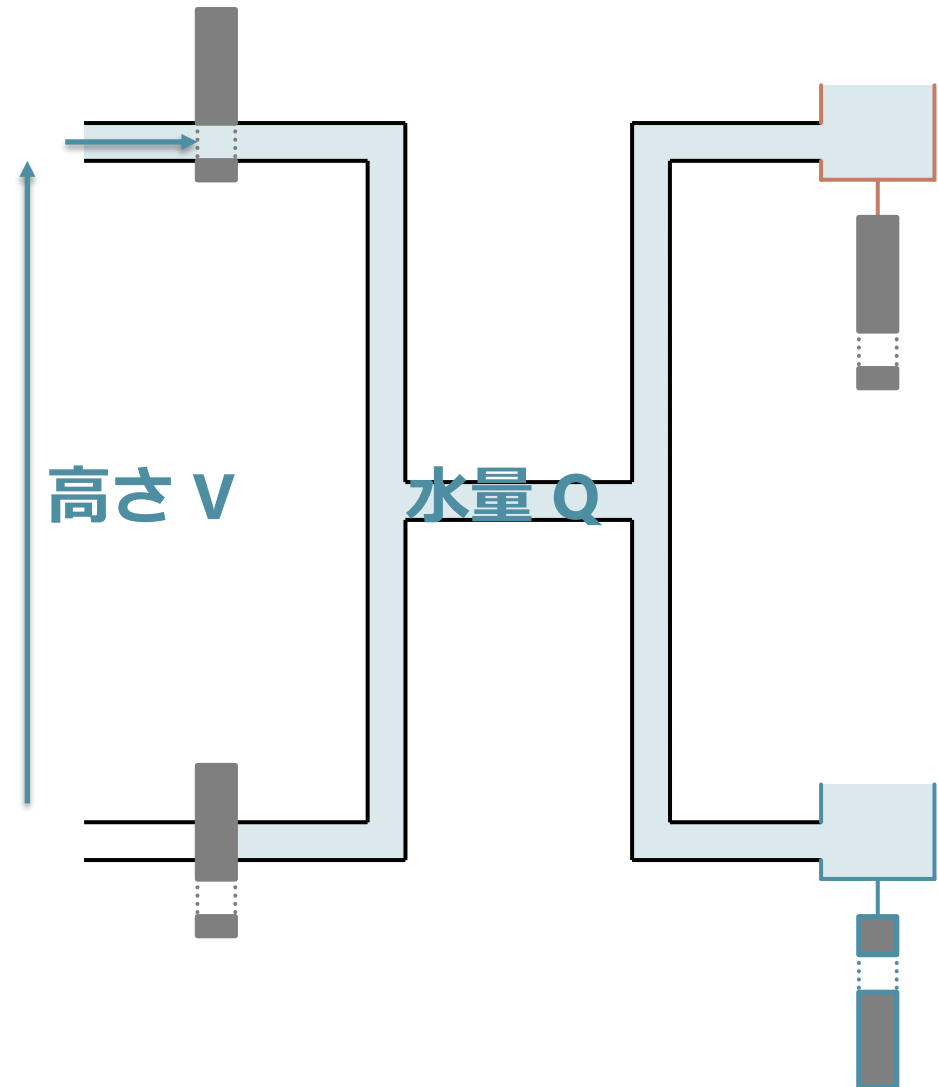
入力が 0 → 1 に遷移すると、
中央と右側タンクの水（斜線部分）が排水される



- 入力が水がくると、右側の2つのタンクにあった水が排水される

つまり、スイッチの切り替えが起きると・・・

- スイッチングが起きると・・・
 - 高さ V の給水源から水量 Q が供給
 - 総量 Q の水がタンクとパイプに入って抜ける
- 入力の変化に伴い、水の出し入れが生じるとエネルギーが消費される
 - 電荷が電源から GND に流れるのも同じ
- 水を高いところから流すとその分位置エネルギーが失われる
 - タンクにたまった水の総量： Q
 - 排水口から給水口高さ： V
 - エネルギー： $Q \times V$



補足：スイッチの ON/OFF 以外による消費エネルギー

- その他に、リーク電流と呼ばれるものもある
 - トランジスタを OFF にしていても、流れ続けてしまう電流
- 分類：
 - 充放電によるもの：動的（dynamic）消費電力
 - リークによるもの：静的（static）消費電力
- リークは、栓が完璧ではないのでチロチロ水が漏れているイメージ

水流システムのアナロジーから説明できること

■ 消費エネルギー：

- 回路：全 PMOS/NMOS のコンデンサに溜まる電荷量 Q と、電圧 V によって決まる
- 水流：全タンクにたまる総水量 Q と、高さ V によって決まる

■ 回路量に比例して消費エネルギーは増える：

- 水流：全タンクに溜まる総水量はタンクの数に比例する

■ 配線も電力を消費する：

- 回路：配線が持つ寄生容量に充放電が起きる
- 水流：パイプにも水は多少溜まる

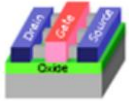
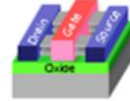
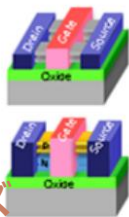
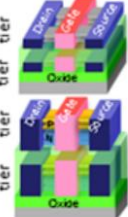
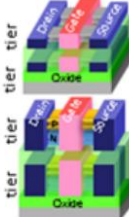
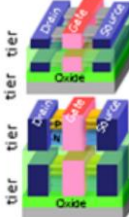
■ 半導体化が微細化すると電力や速度も向上する：

- 回路：コンデンサに電荷をためる量と時間が減る
- 水流：タンクが小さくなるので、水が貯まる時間と量が減る

これらの話は、基本的には半導体技術が進んでも変わらない

図は IEEE INTERNATIONAL ROADMAP FOR DEVICES AND SYSTEMS (IRDS) EXECUTIVE SUMMARY 2022 より

モデル名

YEAR OF PRODUCTION	2022	2025	2028	2031	2034	2037
Logic industry "Node Range" Labeling	G48M24	G42M20	G42M16	G40M16 T2	G38M16 T4	G38M16 T6
Fine-pitch 3D integration scheme	Stacking	Stacking	Stacking	3DVLSI	3DVLSI	3DVLSI
Logic device structure options	finFET LGAA	LGAA	LGAA CFET-SRAM	LGAA-3D CFET-SRAM	LGAA-3D CFET-SRAM	LGAA-3D CFET-SRAM
Platform device for logic	finFET	LGAA	LGAA CFET-SRAM	LGAA-3D CFET-SRAM-3D	LGAA-3D CFET-SRAM-3D	LGAA-3D CFET-SRAM-3D
						

実際のサイズ

LOGIC DEVICE GROUND RULES						
Mx pitch (nm)	32	24	20	16	16	16
M1 pitch (nm)	32	23	21	20	19	19
M0 pitch (nm)	24	20	16	16	16	16
Gate pitch (nm)	48	45	42	40	38	38
Lg: Gate Length - HP (nm)	16	14	12	12	12	12
Lg: Gate Length - HD (nm)	18	14	12	12	12	12
Channel overlap ratio - two-sided	0.20	0.20	0.20	0.20	0.20	0.20
Spacer width (nm)	6	6	5	5	4	4
Spacer k value	3.5	3.3	3.0	3.0	2.7	2.7
Contact CD (nm) - finFET, LGAA	20	19	20	18	18	18
Device architecture key ground rules						
Device lateral pitch (nm)	24	26	24	24	23	23
Device height (nm)	48	52	48	64	60	56
FinFET Fin width (nm)	5.0					
Footprint drive efficiency - finFET	4.21					
Lateral GAA vertical pitch (nm)		18.0	16.0	16.0	15.0	14.0
Lateral GAA (nanosheet) thickness (nm)		6.0	6.0	6.0	5.0	4.0
Number of vertically stacked nanosheets on one device		3	3	4	4	4
LGAA width (nm) - HP		30	30	20	15	15
LGAA width (nm) - HD		15	10	10	6	6
LGAA width (nm) - SRAM		7	6	6	6	6
Footprint drive efficiency - lateral GAA - HP		4.41	4.50	5.47	5.00	4.75
Device effective width (nm) - HP	101.0	216.0	216.0	208.0	160.0	152.0
Device effective width (nm) - HD	101.0	126.0	96.0	128.0	88.0	80.0
PN separation width (nm)	45	40	20	15	15	10

- CMOS の作り方は今後色々変わっていく
 - 色々な3次元構造が導入される予定
 - チップ上にトランジスタを積み上げる事で数を増やす
- 作りが変わっても，電力や速度の考え方は基本的に変わらない

Notes: Mx—Tight-pitch routing metal interconnect IDM—independent device manufacturer FinFET—fin field-effect transistor LGAA—lateral gate all around EUV—extreme ultraviolet NA—numerical aperture Ge—germanium SiGe—silicon germanium RMG—replacement metal gate VLSI—very large-scale integration W2W—wafer to wafer D2W—die to wafer Mem-on-Logic—memory on logic

Figure ES28

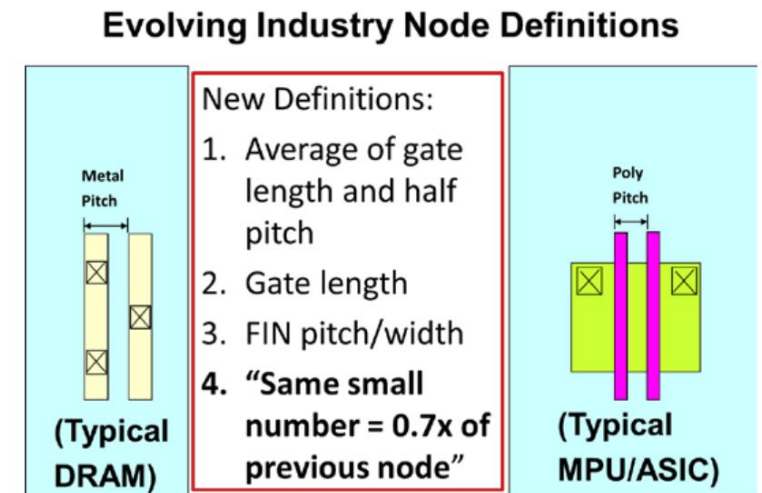
Devices will continue to aggressively scale in the next 5 years

(余談 1) 微細化による半導体世代の「定義の進化」

- 半導体の世代を表す「2nm」などの言葉は「進化」してきた
 - 最初はちゃんと物理サイズを表していた
 - いろいろあって、前の世代の数字に単に 0.7 をかけたものになった
 - 世代の数字が予定通り進まないで格好がつかないため
 - 注意深く見ると企業は「2nm」とは言わず「2N」「2 ナノ世代」等と表記している
- 「現在の半導体は 2nm 程度のサイズであり、これは原子数個分で～」のような表現
 - 「2nm の半導体」は、実際には別にどこも 2nm ではない
 - ただのモデルナンバーなので、そこまで縮んでない
 - 物理的にはまだ 14nm ぐらい
 - 原子サイズの限界まではまだある程度余地がある

図と説明は IEEE INTERNATIONAL ROADMAP FOR DEVICES AND SYSTEMS (IRDS) EXECUTIVE SUMMARY 2022 より。以下は日本語版。

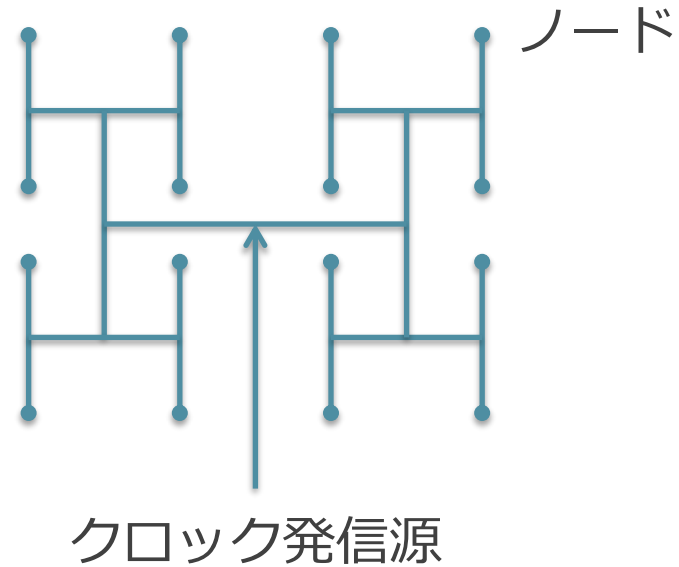
https://irds.ieee.org/images/files/pdf/2022/2022IRDS_ExecutiveSummary_JP.pdf



(余談2) クロックによる消費エネルギー

- クロック信号：
 - 回路の動作の同期をとるための信号で，回路全体に配られる
 - したがって，クロック供給のための配線は長大
 - 配線も寄生コンデンサを作るので，そこで充放電が起きる
- クロック信号線では，配線の総延長がすごいことになる
 - すべての部分が単一のクロック発信源まで接続されている
 - クロック発信源から各部まで配線長が等しくなるように配置
- ただ同期を取るためだけに，チップ内の電力の数割が消費されている
 - クロック信号線全体に電荷を入れて抜くを繰り返している

クロックの配線方法の例：H-TREE



■ H-TREE

- ◇ クロック発信源から各ノードへの配線長が全て等しくなる
- ◇ しかしその分、物理的に近いノードであっても遠回りになる

H-TREE による配線の例 : IBM Power PC

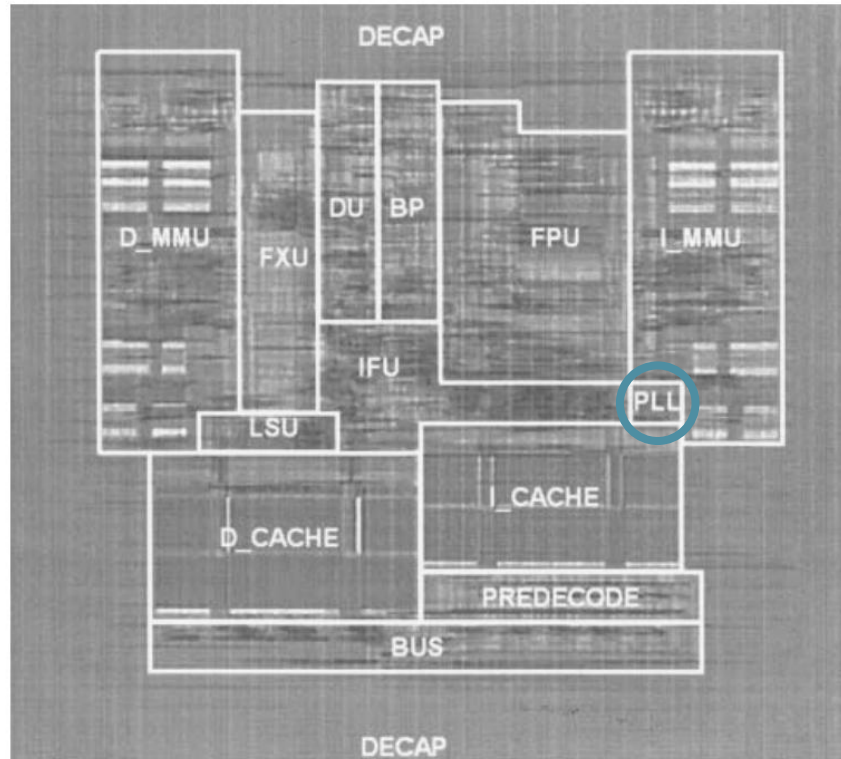


Figure 5.4.2: Die micrograph and floorplan.

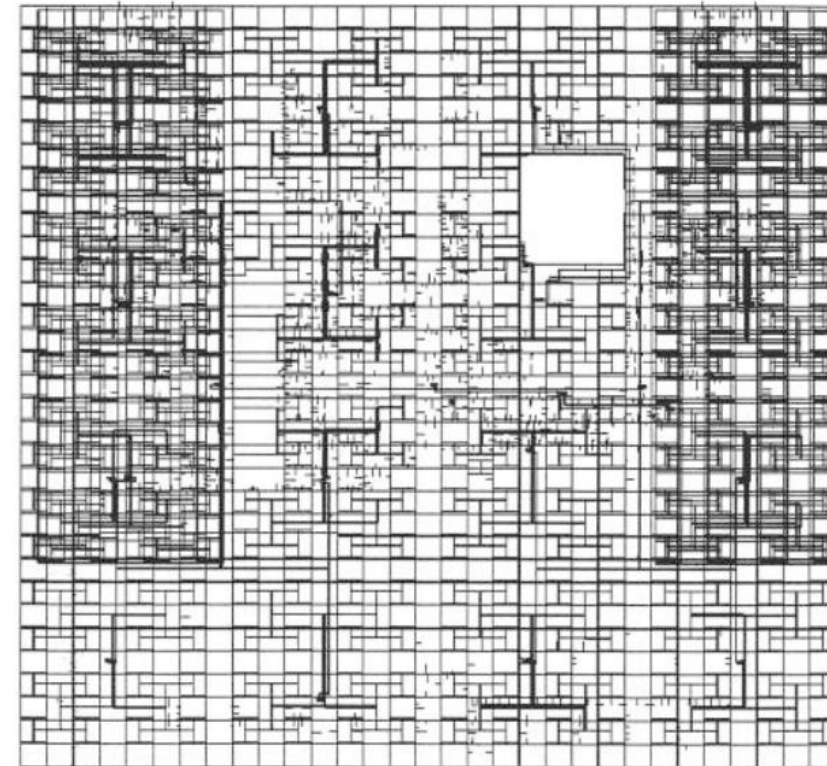


Figure 5.4.7: Clock distribution.

- P. Hofstee, N. Aoki, D. Boerstler, P. Coulman¹, S. Dhong, B. Flachs, N. Kojima, O. Kwon, K. Lee, D. Meltzer², K. Nowka, J. Park, J. Peter, S. Posluszny, M. Shapiro³, J. Silberman², O. Takahashi, B. Weinberger, MP 5.4 A 1GHz Single-Issue 64b PowerPC Processor, ISSCC 2000 より

図は以下より：

KIM, Youngchan; KIM, Taewhan. Algorithm for synthesis and exploration of clock spines. In: 2017 22nd Asia and South Pacific Design Automation Conference (ASP-DAC). IEEE, 2017. p. 263-268.

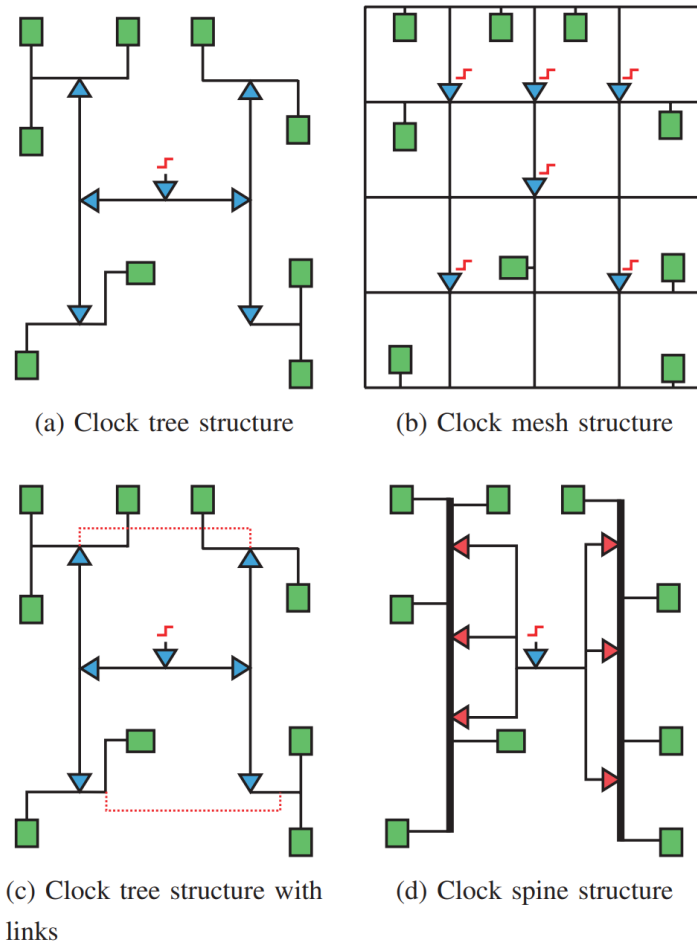
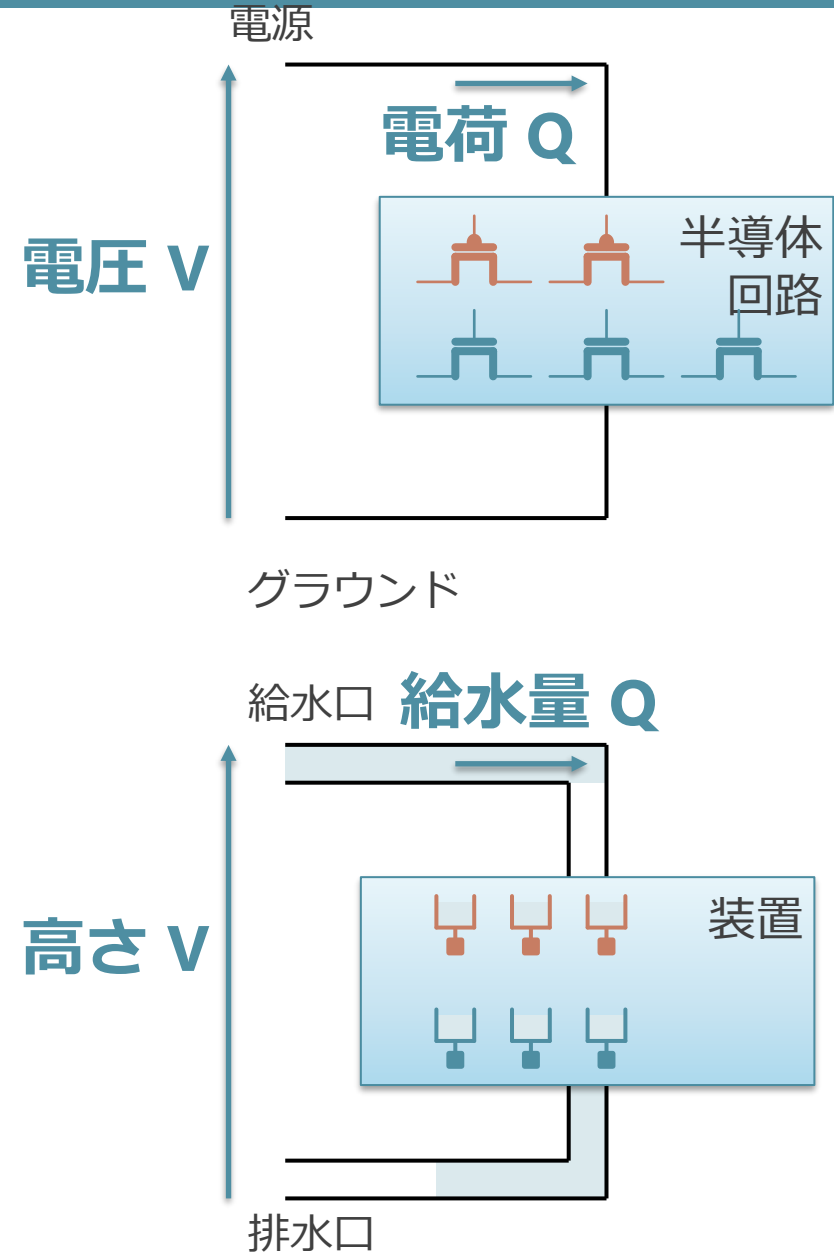


Fig. 1. Two extreme clock network structures are shown in (a) and (b). Two intermediate clock network structures are shown in (c) and (d). Clock spine structure in (d) delivers clock signal from clock source to clock sinks through top-level clock tree, clock spines, and stubs. At least two spine buffers have to be placed on each clock spine to maintain the tolerance to clock skew variability.

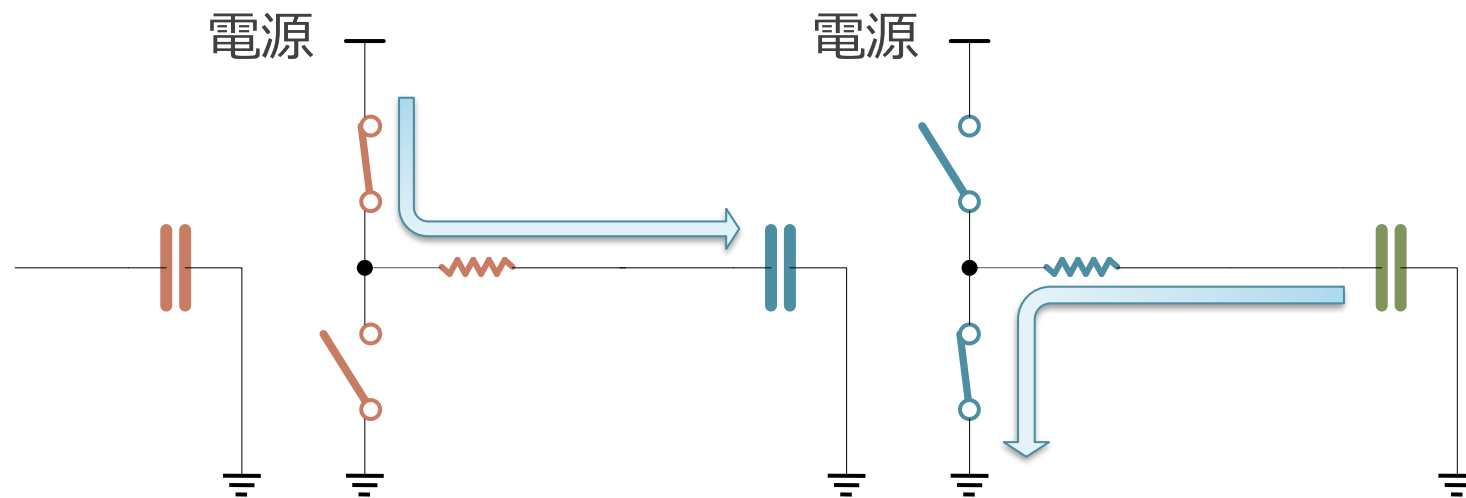
- skew を無くすためには、クロックネットワーク全体で電圧のズレがなければ良い
- clock grid：メッシュ状の配線を使う
 - ノード同士を密に縦横で繋いでやれば電位差が生まれにくい
- clock spine：太い配線を幹にして、そこから配る
 - 太い線の中では差が生まれにくいので、そこから配る

ここまでのまとめ

- CMOS 半導体の消費電力：
 - 電荷の充放電と電圧により主に決まる
 - システムに流した水の量と高さのイメージ
- 消費電力は回路の大きさに比例する
 - 充放電される電荷の量は，回路の量に比例
 - 水タンクの総数や総量が大きくなるイメージ
- 消費電力は動作周波数にも比例する
 - 充放電回数が比例するから
 - 水を入れて抜く回数が増えるとエネルギーが増えるイメージ



CMOS ゲートの遅延の実体



■ 遅延：コンデンサの充放電にかかる時間

1. あるゲートのスイッチが切り替わる
2. 次の段のゲートへの充放電が開始
3. 次の段のスイッチが切り替わる
4. ...

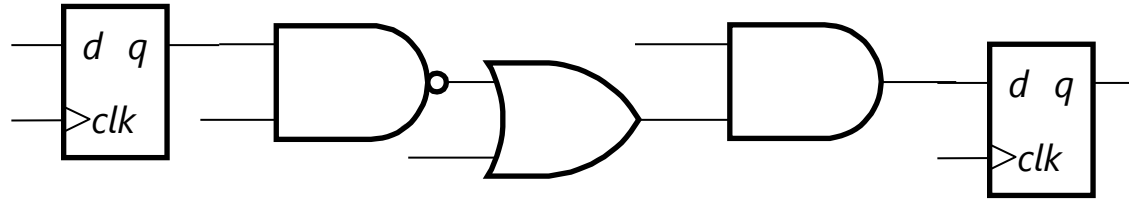
■ 寄生容量：

- トランジスタのゲート部分以外にも、あらゆる場所にコンデンサができてしまう
- なにか導体が不導体をはさんで並べばコンデンサになる

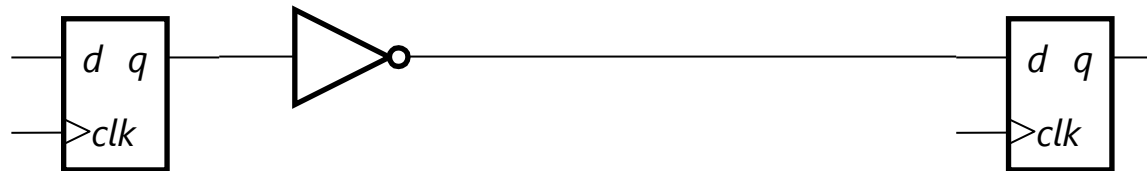
■ 寄生容量への充放電にも時間がとられる

- 通常はトランジスタのゲートがメイン
- 長い配線では、配線間にできてしまうコンデンサの影響も大きい

遅延の違い

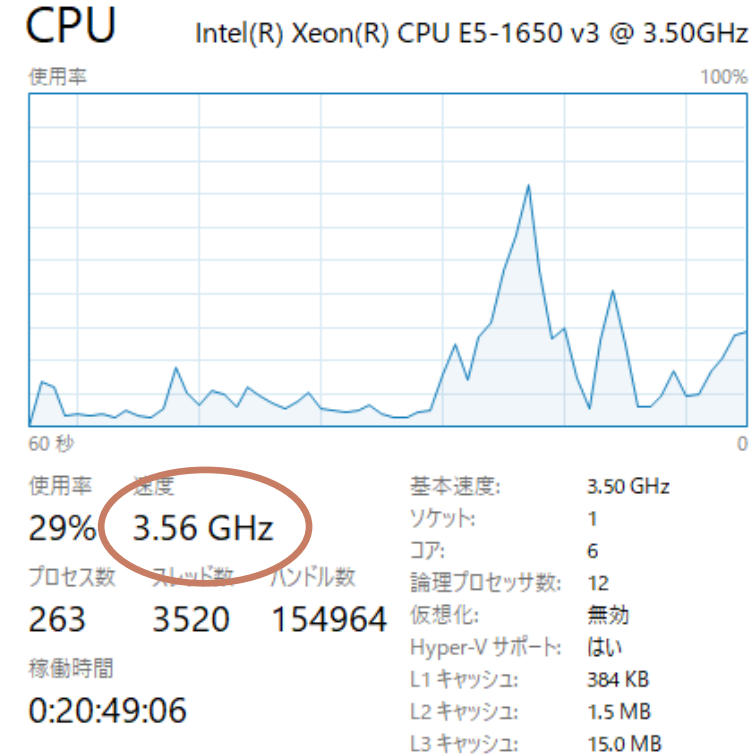
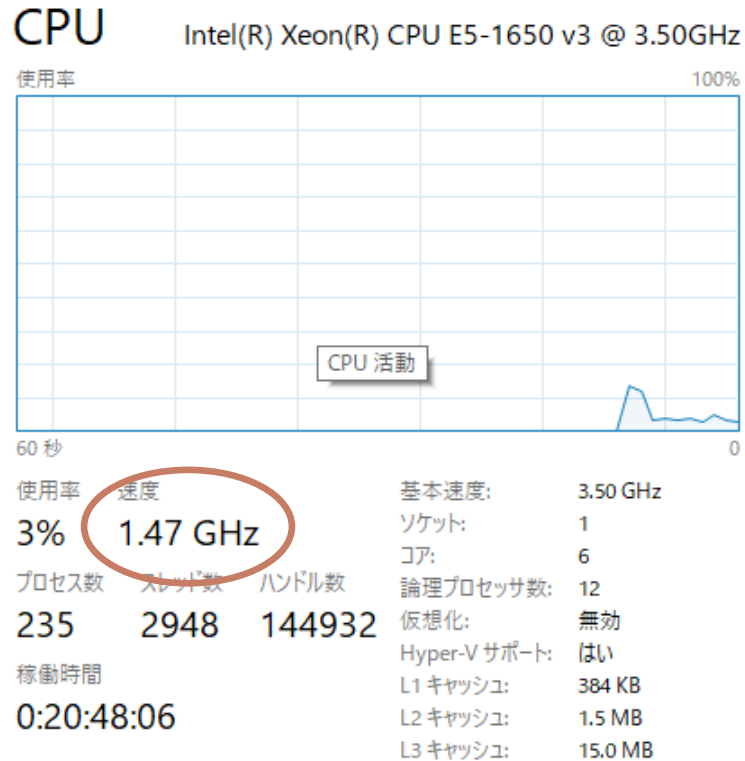


- 左側の D-FF から何個もゲートを経て右の D-FF に接続（遅い）
 - q が切り替わると,
 - 各ゲートのスイッチ（充放電）が順々に行われて,
 - 右側の d に伝わる



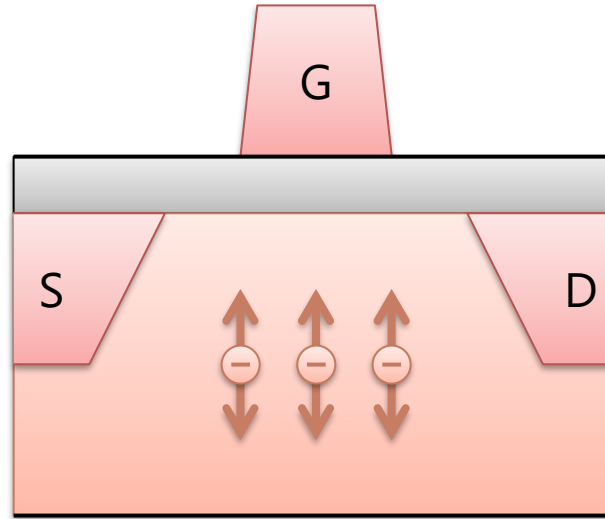
- 左側の D-FF から 1 つのゲートだけを経て右の D-FF に接続（速い）
 - ◇ q が切り替わると, 1 回だけスイッチ（充放電）が行われて,
 - ◇ 右側の d に伝わる

余談 : DVFS (Dynamic Voltage Frequency Scaling)



- CPU は通常、負荷に応じて動作周波数を変えている
 - 消費電力削減のため
 - この時、電圧も同時に操作している

電圧と遅延



- Gate にかける電圧を上げると、より高速に動作する
 - よりたくさん電子が上に集まる
 - チャンネルが厚くなり、電流が流れやすくなる
 - 次段のコンデンサの充放電が速くなる

DVFS: Dynamic Voltage Frequency Scaling

- 電圧と周波数の組を用意しておき, これを切り替える
 - 高速 : 高周波数 (低遅延) で動かすために, 電圧を上げる
 - 低速 : 低周波数 (高遅延) でよいので, 電圧を下げる
- 電圧を下げると, すごく消費電力が減る
 - 消費エネルギーは電圧の2乗に比例 : $E = CV^2$
- 低周波数にすると充放電の回数自体 (=周波数) も減る
 - 結果として, 3乗のオーダーで電力が削減できる

まとめ

- 目的：これらの具体的なイメージを持つ
 - CPU の論理的な動作と，それを実現する物理的な回路の繋がり
 - それら論理回路の遅延
- 論理回路の復習から始めて，遅延が何でできるのかまでを説明
 - 論理回路と，その設計
 - CMOS による実現
 - 遅延
- この講義資料では（ていうか，今後の資料も結構），一部，五島先生の「デジタル回路」の講義資料の図を使用しています

出欠と感想

- 本日の講義でよくわかったところ, わからなかったところ, 質問, 感想などを UTOL の出席欄に書いてください
 - LMS の出席を設定するので, そこにお願いします
 - パスワード : arc
- 意見や内容へのリクエストもあったら書いてください
- UTOL の出席の締め切りは来週の講義開始までに設定してあります
 - 仕様上「遅刻」表示になりますが, 特に減点等しません
 - 来週の講義開始までは感想や質問などを受け付けます

出欠について

- 来週から本当に現地で出欠をとります

質問や感想

質問と回答とか

- 今日はCPU関連の話がありましたが、今まで僕が勝手に思ってるものと結構違っていたところにびっくりしています。特にC言語との関連性の話について、昔計算物理学という授業を受けていたときに、「C言語やFortranは、コードを書くのは大変だが計算がすっきりしているから計算屋さんは好んで使う」という教授の話を思い出す。
- 僕自身は天体物理など(前回コメント紹介では天文台の方の話も結構共感でした)を数値シミュレーションで再現して物理を議論する研究をしています。僕の場合、N体問題というよりも流体計算をやることが多く、そこでGPUを用いた並列計算のコードを開発したいと思い出すこの授業をとりました。業界既存のコードは基本的にCPUを用いるので、そこは時代に合わせたGPU計算をやってみたいという背景があります。
- この授業の過去スライドではコード書く話はあまりないのですが、コードと計算機の相性を知るためにもこの授業を頑張っで行こうかなと思っています。

- 僕は専用ハードウェアの設計を専門としている研究室に所属していますが、そこにも最終的にはRISC-Vを乗せるので、非常に親近感のある講義でした。RISC-Vの実際の実装例も、聞いたことある企業でRISC-Vの勢いを感じました。

- 最近x86基板のカスタムOSを趣味で作っていて、授業でx86でなんか挙げるとまずいと言われてびっくりしました。ですが、それは命令セットの話ですよ、自分のはQEMUで動かしているので多分大丈夫だと思います。

- ちなみに、研究室の研修でRISC-Vの設計をさせているとおっしゃいましたが、どこまで作っているのか気になります。（ファウンドリーレベル？）

- RISC-Vでは浮動小数点や乗除算も必須部分には含まれずに拡張になっているということが直感的には意外でしたが、確かに設計としてはそれが良いのかなと思いました。

- 以前学んだRISC-Vの立ち位置がよくわかって面白かった。x86やArmが権威主義的・保守的でRISC-Vがオープンソース志向であることは、OSにおけるWindows, MacOSとLinuxの違いのようなものなのだなと思った。

- RISC V 32IのCPUを設計したことがあるが、あれだけの命令でとてもOSなどを動かすことはできないように感じたので必須の命令セットがどのような基準で設定されているのかが気になった。

- 機械系＞ 知能機械＞ 創造情報なので、コンピューターアーキテクチャに関しては自分の興味で調べた以外は詳しくないです。ロボットとAIの研究していて最近研究インターンなどでCUDAカーネルやハードレベルでの最適化が必要そうなところが出てきたのでここで学んだ知見を活かさせたいと思います。

- 学部時代の実験でRISC-Vのプロセッサを作ったのですが、その頃から意味不明だと思っていた命令のbit配置の謎が少し解けたのが面白かったです。(命令ごとに即値の配置がぐちゃぐちゃなのが一番の謎だったので、割愛されてしまったのは残念でしたが...)

- C拡張で使用空間が75%減るというのはそれはそうだが、C拡張を外してまで今すぐ入れたい命令があるのか気になった。

- 【質問】 今後RISC-Vを活用して開発する場合に役立つと思われる、推薦図書または参照資料はありますか。講義の後半部に、『RISC-V 原典：オープンアーキテクチャのススメ』の写真があったかと思いますが、この本とRISC-V 公式の仕様書以外に有益なソースがあればご教示いただきたいです。

- RISC-Vの使用を決めた人(Andrew Waterman)の博論が2016年に提出されているとのことで、意外と最近だなと感じた。

- 私は専門が情報科学系ではなく、独学でコンピュータ・アーキテクチャについて勉強していました。(ゲーム ""Turing Complete""でCPUを作るなど.....)今回の復習講義は、自分の認識が情報科学の人たちとそこまで大きくはずれていないようで良かったと感じられました。